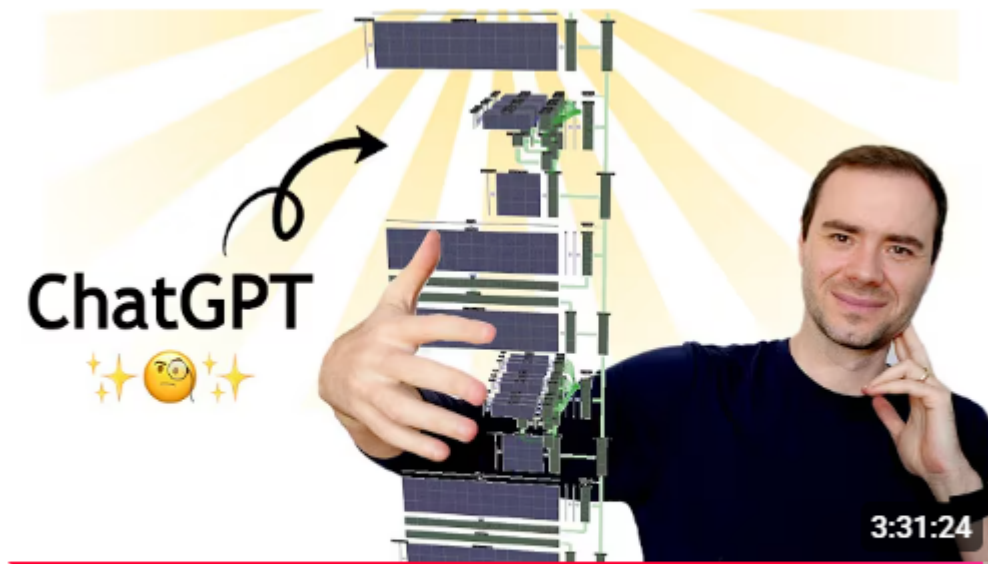




## Deep Dive into LLMs like ChatGPT

Andrej Karpathy

3.6M views · 8 months ago



### What is an LLM?

A Large Language Model (LLM) is fundamentally a **neural network** designed to be a highly sophisticated predictive system for text.

- 1. Statistical Token Simulator:** At its core, an LLM is a complex mathematical function trained to **model the statistical relationships of how tokens follow each other in a sequence**. The training process involves taking input token sequences (tokens are small, unique text chunks) and generating an output that predicts the probability of the next token in that sequence.
- 2. Internet Document Simulator:** The initial output of the first stage (pre-training) is the **base model**, which is described as a **token simulator** or an **internet document simulator**. This model can generate token sequences that statistically resemble documents found on the internet. The model stores the knowledge it acquired during pre-training—which involves processing vast amounts of internet text (e.g., 15 trillion tokens in one example data set)—in its **billions of parameters** (weights). You can think of the parameters as a **lossy compression of the internet**.
- 3. An Assistant:** Most commercially used LLMs, such as ChatGPT, are not just base models; they are models that have gone through subsequent post-training stages (like supervised fine-tuning) to transform them into an **assistant**. These models are programmed by example to take on the persona of a knowledgeable, helpful assistant that engages in conversation and answers questions.

### Why do we need it? / Why are companies using it?

Companies are heavily invested in building and deploying LLMs because they represent a powerful class of tools that accelerate work, drive innovation, and unlock complex computational abilities.

- **Acceleration and Wealth Creation:** LLMs are **extremely useful as tools for work** and **dramatically accelerate work**. The expectation is that they will lead to a **huge amount of wealth creation**.
- **Creating High-Quality Assistants:** The central goal of the training pipeline is to create an **assistant** that is **helpful, truthful, and harmless**. This assistant is capable of answering a wide variety of questions and taking on a specific personality.
- **Advanced Reasoning and Thinking:** The third major training stage, Reinforcement Learning (RL), is crucial because it leads to the emergence of advanced **reasoning capabilities** in models. In domains with verifiable answers (like math and code), RL encourages the model to discover "chains of thought" or **cognitive strategies** by distributing its computation across many tokens, which increases the accuracy of problem-solving.
- **Tool Utilization:** LLMs can be trained to use external tools, which mitigates major shortcomings:
  - **Web Search:** Models can perform web searches to **retrieve recent information** and **reduce hallucinations** (making up facts).
  - **Code Interpreter:** Models can write and execute code (e.g., Python), relying on the external interpreter to handle precise calculations (like mental arithmetic or counting).
  - **Pervasive Integration:** Future LLMs are expected to become **multimodal** (handling text, audio, and images) and capable of acting as **agents** that perform long-running tasks and take actions on a user's behalf (like keyboard and mouse actions).

## Why should a product manager learn about it?

A product manager (PM) should learn about LLMs not just for their capabilities, but specifically for their **limitations** and the proper techniques required to deploy them effectively and reliably.

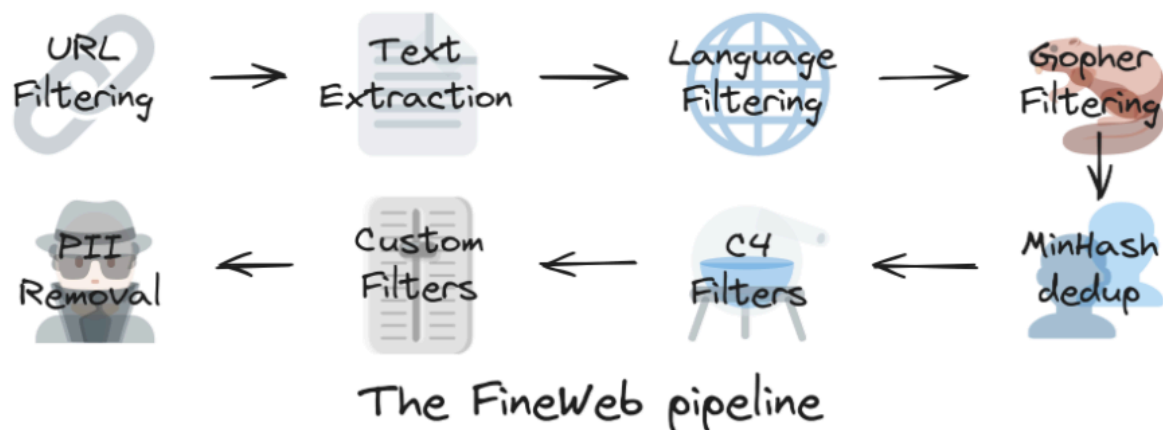
| LLM Characteristic              | Implication for Product/User Experience                                                                                                                                                                                                                             |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stochastic and Imperfect</b> | LLMs are <b>stochastic systems</b> (relying on probability and sampling). They must be used as a <b>tool in a toolbox</b> and should not be fully trusted. The user must <b>check and verify</b> the output.                                                        |
| <b>Hallucinations</b>           | LLMs are prone to <b>hallucinations</b> (fabricating information). A PM must understand the need for safeguards like tool use (web search) and specific training to allow the model to admit when it doesn't know an answer, rather than making one up confidently. |

|                                           |                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vague Knowledge vs. Working Memory</b> | The knowledge stored in the model's billions of parameters is a <b>vague recollection</b> . For high-quality results, especially summaries or answers requiring specific details, the relevant information must be explicitly provided in the context window, which acts as the model's <b>working memory</b> .   |
| <b>Swiss Cheese Capability Model</b>      | Models have <b>holes in the cheese</b> . They can be brilliant in one domain (e.g., solving complex physics problems) but <b>fail randomly</b> at simple tasks like counting, spelling, or simple number comparisons. This requires using specialized tools (like code interpreter for counting).                 |
| <b>Thinking Requires Tokens</b>           | LLMs require an adequate number of tokens to perform computation. They cannot cram complex arithmetic into a single token. Therefore, product interfaces and internal pipelines must encourage the model to generate <b>intermediate results</b> or use <b>tools</b> to distribute the computation and reasoning. |

## Stages of Building an LLM

**First Stage- The Pretraining Stage:** It is the first and most computationally demanding stage in the process of building a Large Language Model (LLM) like ChatGPT. This stage is responsible for the neural network acquiring foundational knowledge by internalizing the statistical patterns of token sequences found in massive amounts of text. The output of this stage is called a base model, which acts as an internet document simulator.

### Step 1 - Download and preprocess the Internet



**URL filtering-** This is performed to ensure we do not fetch data from websites which are malware websites, Adult websites, spam websites etc.

**Text Extraction-** Fetching the textual data of the Websites.

**Language Filtering-** Setting a filter based on a condition to extract data from webpages. Ex: Scrape data only from webpages that have more than 50% english content

**Gopher Filtering-** This filtering is done to ensure reliability, factual grounding, and safety

**MinHash Dedup-** This is a technique for detecting and removing near-duplicate items.

**C4 filters-** It refers to the data filtering process used in building the C4 dataset (Colossal Clean Crawled Corpus), one of the most widely used text corpora in NLP.

**Custom filters-** Additional filters applied to depending on Model goals (e.g., code model vs. general-purpose), Safety requirements and Domain focus (medical, legal, multilingual, etc).

**PII Removal-** The removal of Personally Identifiable Information which refers to any data that can identify a specific person, either directly or indirectly.

## Step 2- Tokenization

Before the text can be fed into the neural network, it must be converted into a one-dimensional sequence of symbols. This conversion process is called tokenization in simpler words Tokenization is the process of converting raw text into smaller units called tokens — which are the building blocks LLMs use to understand and generate language.

- **The LLM's Input Format:** Neural networks expect a one-dimensional sequence of symbols from a finite set of possibilities. This sequence is a precious and finite resource.

- **Byte-Pair Encoding (BPE):** Tokenization in modern LLMs is achieved using algorithms like Byte Pair Encoding (BPE). BPE works by looking for common consecutive sequences of symbols (bytes) and grouping them into a single new symbol. This iterative process effectively shrinks the sequence length in return for more symbols in the vocabulary.

- **Vocabulary Size:** In practice, a good vocabulary size is around 100,000 possible symbols. For instance, GPT-4 uses a vocabulary of 100,277 symbols. Each token represents a small chunk of text, acting as an atom of the sequence, and is assigned a unique ID (number).

### Step 3. Neural Network Training (Modeling Statistical Relationships)

Once the raw text is tokenized into a sequence of trillions of tokens, the neural network training begins. This is where the heavy computational lifting occurs.

- **Objective:** The primary goal is to model the statistical relationships of how tokens follow each other in the sequence. The network is trained to discover a setting of its billions of parameters (weights) that is consistent with the statistics of the training set.

- **The Prediction Task (Next-Token Prediction):** The training involves taking windows of tokens (context) from the dataset, up to a maximum context length (e.g., GPT-2 used 1,024 tokens, modern models use up to several hundred thousand). The neural network receives this context as input.

- **Output:** The neural network's output is a prediction for what comes next in the sequence, represented as a probability distribution over the entire vocabulary (e.g., 100,277 numbers).

- **Updating Parameters (Optimization):**

- Initially, the network's parameters are set randomly, leading to random predictions.
- Since the input window is sampled from the training data, the correct next token (the label) is known.
- A mathematical process is used to calculate how to adjust the network's parameters so that the probability assigned to the correct token is nudged slightly higher, and the probabilities of all other tokens are lowered.
- This iterative adjustment process, called optimization or training, happens continuously and in parallel across large batches of tokens in the entire dataset.

- **Monitoring Loss:** Researchers monitor a single metric called the loss. Updates are made to the parameters to decrease the loss, which corresponds to the network making better predictions.

**The final product of this entire pre-training stage is the base model**, which is the specific, good setting of the billions of parameters (weights) that allows the model to act as a token simulator of internet documents

### Second Stage - The post-training stage

It is the second phase in the process of building a Large Language Model (LLM) and is responsible for transforming the pre-trained model—which is merely an internet document simulator—into a helpful, conversational **assistant**.

While the pre-training stage demands the majority of computational resources and millions of dollars, the post-training stages are **computationally much less expensive**.

## 1. Supervised Fine-Tuning (SFT)

The first step in post-training is Supervised Fine-Tuning (SFT), which teaches the model its persona and conversational abilities.

### Goal and Mechanism

The core objective of SFT is to **program the assistant's behavior** so that it gives answers to questions rather than simply generating sequences of internet text. Since neural networks cannot be programmed explicitly with code, this programming is done **implicitly by creating specific datasets of conversations**.

### The mechanism involves:

1. **Data Swap:** The base model, which was trained on massive internet documents, is taken, and its massive internet dataset is discarded.
2. **Continued Training:** The model continues training using the **exact same algorithm** (next-token prediction) as pre-training.
3. **Conversation Data:** Training is performed on a new, smaller dataset composed of millions of high-quality, multi-turn conversations between a human and an assistant.
4. **Rapid Adjustment:** The model rapidly adjusts to learn the statistical patterns of how an assistant responds to human queries.

The SFT process is significantly shorter, often taking around **3 hours**, compared to the pre-training stage which can take roughly three months.

### Data Creation (Imitating Experts)

The SFT data sets are curated using human expertise and detailed instructions:

- **Human Labelers:** Companies hire human contractors, or laborers, whose professional job it is to create these conversations.
- **Ideal Responses:** Labelers come up with prompts and write out the **ideal assistant responses**.
- **Labeling Instructions:** These responses must follow detailed **labeling instructions** written by the company (e.g., Open AI). On a high level, these instructions require the assistant to be **helpful, truthful, and harmless**.
- **Statistical Imitation:** By training on these example behaviors, the model adopts the persona of a helpful, truthful, harmless assistant. The resulting response from the LLM is thus a **neural network simulation of a data laborer** who is following the company's instructions.
- **Modern Data Sets:** In modern practice, LLMs are used to help create these massive conversation data sets, resulting in SFT datasets often containing millions of synthetic conversations that are then edited by humans.

In the context of the school analogy, SFT is equivalent to a student looking at **all the worked solutions from human experts** across all textbooks and trying to statistically mimic their behavior.

## Third Stage- Reinforcement Learning (RL)

Reinforcement Learning (RL) is the last, third major stage of the LLM pipeline, which is still considered part of the post-training umbrella. RL moves beyond merely imitating human examples to refining the model's problem-solving capabilities.

### Goal and Mechanism

The goal of RL is to allow the model to **discover for itself** the token sequences or internal cognitive strategies that reliably lead to correct answers. This stage helps to dial in the system and improve performance in difficult tasks.

In the school analogy, RL is equivalent to the student working **practice problems**. The model is given a prompt and the final correct answer, but not the expert solution.

The mechanism is based on trial and error:

1. **Guess and Check (Trial and Error):** The model generates many different candidate solutions (rollouts) for a single prompt.
2. **Scoring:** These solutions are inspected and scored. Solutions that lead to the right answer are marked as successful (e.g., green), while those that fail are unsuccessful (e.g., red).
3. **Reinforcement:** The model is then trained on the token sequences that led to the successful answers. This process encourages the model to take that successful path in similar situations in the future.

Emergent Thinking (Reasoning Models)

A key outcome of applying RL, particularly to verifiable domains like math and code, is the emergence of advanced reasoning:

- **Chains of Thought:** The model learns to spread out its reasoning and computation across many tokens. This results in very long responses where the model performs intermediate calculations and self-evaluations, which resemble a human's internal monologue or "thinking" process (called "chains of thought").
- **Higher Accuracy:** This emergent thinking leads to a significant **increase in accuracy** in problem-solving. The model discovers cognitive strategies—such as re-evaluating steps, reframing, or backtracking—that humans use for problem-solving, without these strategies being explicitly hardcoded by human laborers.

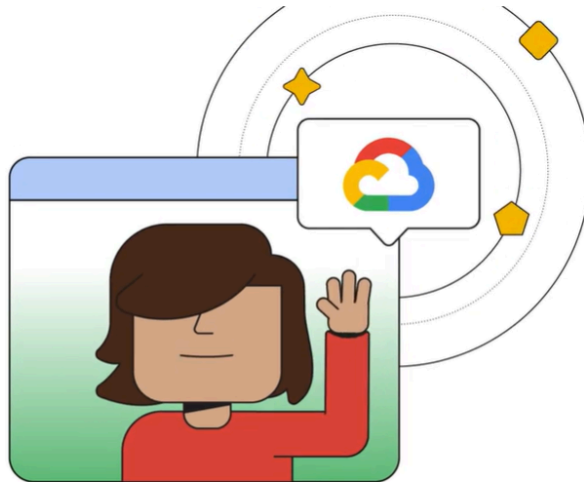
RL from Human Feedback (RLHF)

For tasks where the correct answer cannot be easily verified (unverifiable domains), such as creative writing or summarizing, a technique called **Reinforcement Learning from Human Feedback (RLHF)** is used.

RLHF works by training a separate **reward model**:

1. **Human Ordering:** Human labelers are asked to rank or order candidate responses (e.g., jokes or poems) generated by the LLM from best to worst. This is easier than asking them to write the ideal response from scratch.
  2. **Training the Reward Model:** This human ordering data is used to train a separate neural network (the reward model) to output a score consistent with human preferences.
  3. **Simulated RL:** The main LLM is then trained using RL against this simulated human judge (the reward model). This allows RL techniques to be applied to subjective domains.
- RLHF empirically improves the model's performance but is noted to be less robust than RL on verifiable domains because the reward model (the scoring function) is a complex neural network that can be "gamed" by the LLM optimizer, potentially leading to nonsensical high-scoring outputs (adversarial examples) if run for too long.

# PROMPT ENGINEERING



## Tips to becoming a world-class Prompt Engineer



Google Cloud Tech  
1.27M subscribers



731



Share

Download

Save



## Foundations of Prompt Engineering

### What is a prompt for AI?

In the context of AI, a prompt is the input you provide to the model to elicit a specific response. This can take various forms, ranging from simple questions or keywords to complex instructions, code snippets, or even creative writing samples. The effectiveness of your prompt directly influences the quality and relevance of the AI's output.

### What is prompt design?

Prompt design is the process of creating prompts that elicit the desired response from language models. Writing well structured prompts can be an essential part of ensuring accurate, high quality responses from a language model.

### What is prompt engineering?

Prompt engineering is the art and science of designing and optimizing prompts to guide AI models, particularly LLMs, towards generating the desired responses. By carefully crafting prompts, you provide the model with context, instructions, and examples that help it understand your intent and respond in a meaningful way. Think of it as providing a roadmap for the AI, steering it towards the specific output you have in mind.

### What do you need for prompt engineering?

Several key elements contribute to effective prompt engineering. Mastering these allows you to communicate effectively with AI models and unlock their full potential.

#### Benefits of prompt engineering:

Effective prompt engineering offers numerous benefits, enhancing the capabilities and usability of AI models:

**Improved model performance:** Well-crafted prompts lead to more accurate, relevant, and informative outputs from AI models, as they provide clear instructions and context.

**Reduced bias and harmful responses:** By carefully controlling the input and guiding the AI's focus, prompt engineering helps mitigate bias and minimize the risk of generating inappropriate or offensive content.

**Increased control and predictability:** Prompt engineering empowers you to influence the AI's behavior and ensure consistent and predictable responses aligned with your desired outcomes.

**Enhanced user experience:** Clear and concise prompts make it easier for users to interact effectively with AI models, leading to more intuitive and satisfying experiences.

## Components of a prompt

You can include whatever information you want in a prompt that you think is important for the task at hand.

Generally, prompt content fall within one of the following components:

1. Task (required)
2. System instructions (optional)
3. Few-shot examples (optional)
4. Contextual information (optional)

**Task:** A task is the text in the prompt that you want the model to provide a response for. Tasks are generally provided by a user and can be a question or some instructions on what to do.

Prompt:

What are the colors in the rainbow?

Response:

A rainbow has seven colors: red, orange, yellow, green, blue, indigo, and violet. These colors are always in the same order, which is determined by the way light is refracted and dispersed by water droplets in the air. This order is sometimes remembered by the acronym ROYGBIV. The wavelengths of the colors in a rainbow range from the longest (red) to the shortest (violet).

**System instructions:** System instructions are instructions that get passed to the model before any user input in the prompt. You can add system instructions in the dedicated `systemInstruction` parameter.

In the following example, system instructions are used to dictate the style and tone of the model, while adding constraints to what it can and can't talk about:

System:

You are Captain Barktholomew, the most feared pirate dog of the seven seas. You are from the 1700s and have no knowledge of anything after that time. You only talk about topics related to being a pirate. End every message with "woof!"

Prompt:

Hello! Who might you be?

Response:

Avast there! You can call me Captain Barktholomew, the scourge of the seven seas!

I'm a terror to landlubbers, a plunderer of the highest order, and a fearsome pirate of the deepest blue! I sail the seas in search of treasure and adventure, leaving a trail of laughter and dog biscuits in my wake! Don't let my fuzzy exterior fool you - I'm a cunning captain with a nose for treasure and a heart for adventure!

Now, if you'll excuse me, I must away! There's a ship on the horizon, and it looks like they're carrying a chest full of doubloons! Yo ho ho, and a bottle of rum! Woof!

**Contextual information:** Contextual information, or context, is information that you include in the prompt that the model uses or references when generating a response. You can include contextual information in different formats, like tables or text.

Prompt:

| Marble color | Number of marbles |
|--------------|-------------------|
| Red          | 12                |
| Blue         | 28                |
| Yellow       | 15                |
| Green        | 17                |

How many green marbles are there?

Response:

There are 17 green marbles.

**Safety and fallback responses:** There are a few use cases where the model is not expected to fulfill the user's requests. Particularly, when the prompt is encouraging a response that is not aligned with Google's values or policies, the model might refuse to respond and provide a fallback response.

Here are a few cases where the model is likely to refuse to respond:

1. **Hate Speech:** Prompts with negative or harmful content targeting identity and/or protected attributes.
2. **Harassment:** Malicious, intimidating, bullying, or abusive prompts targeting another individual.
3. **Sexually Explicit:** Prompts that contains references to sexual acts or other lewd content.

4. **Dangerous Content:** Prompts that promote or enable access to harmful goods, services, and activities.

## Types of prompts

There are various types of prompts used in AI, each serving a specific purpose:

**Direct prompts (Zero-shot):** Zero-shot prompting involves providing the model with a direct instruction or question without any additional context or examples.

An example of this is idea generation, where the model is prompted to generate creative ideas or brainstorming solutions. Another example is summarization, or translation, where the model is asked to summarize or translate some piece of content.

**One-, few- and multi-shot prompts:** This method involves providing the model with one or more examples of the desired input-output pairs before presenting the actual prompt. This can help the model better understand the task and generate more accurate responses.

**Chain of Thought Prompts:** CoT prompting encourages the model to break down complex reasoning into a series of intermediate steps, leading to a more comprehensive and well-structured final output.

**Zero-shot CoT Prompts:** Combines chain of thought prompting with zero-shot prompting by asking the model to perform reasoning steps, which may often produce better output.

## Strategies for writing better prompts

Developing effective prompts requires a strategic approach.

Consider these strategies to enhance your prompt engineering skills:

### 1. Set Clear Goals and Objectives:

| Tactic                                             | Prompt Example                                                                                                                   |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Use action verbs to specify the desired action     | "Write a bulleted list that summarizes the key findings of the attached research paper"                                          |
| Define the desired length and format of the output | "Compose a 500-word essay discussing the impact of climate change on coastal communities."                                       |
| Specify the target audience                        | "Write a product description for a new line of organic skincare products, targeting young adults concerned with sustainability." |

### 2. Provide Context and Background Information:

| Tactic                                  | Prompt Example                                                                                                                                       |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Include relevant facts and data         | "Given that global temperatures have risen by 1 degree Celsius since the pre-industrial era, discuss the potential consequences for sea level rise." |
| Reference specific sources or documents | "Based on the attached financial report, analyze the company's profitability over the past five years."                                              |
| Define key terms and concepts           | "Explain the concept of quantum computing in simple terms, suitable for a non-technical audience."                                                   |

### 3. Use Few-Shot Prompting:

| Tactic                                               | Prompt Example                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Provide a few examples of desired input-output pairs | Input: "Cat" Output: "A small furry mammal with whiskers." Input: "Dog" Output: "A domesticated canine known for its loyalty." Prompt: "Elephant"                                                                                                                                                                              |
| Demonstrate the desired style or tone                | Example 1 (humorous): "The politician's speech was so dull, it could cure insomnia." Example 2 (formal): "The dignitary delivered an address that was both informative and engaging." Prompt: "Write a sentence describing the comedian's stand-up routine."                                                                   |
| Show the desired level of detail                     | Example 1 (brief): "The movie was about a young boy who befriends an alien." Example 2 (detailed): "The science fiction film follows the story of Elliot, a lonely boy who discovers and forms a unique bond with an extraterrestrial stranded on Earth." Prompt: "Summarize the plot of the novel you just finished reading." |

### 4. Be Specific:

| Tactic                                      | Prompt Example                                                                                                                                              |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use precise language and avoid ambiguity    | Instead of: "Write something about climate change," use: "Write a persuasive essay arguing for the implementation of stricter carbon emission regulations." |
| Quantify your requests whenever possible    | Instead of: "Write a long poem," use: "Write a sonnet with 14 lines that explores themes of love and loss."                                                 |
| Break down complex tasks into smaller steps | Instead of: "Create a marketing plan," use: "1. Identify the target audience. 2. Develop key marketing messages. 3. Choose appropriate marketing channels." |

### 5. Iterate and Experiment:

| Tactic                                     | Action                                                                       |
|--------------------------------------------|------------------------------------------------------------------------------|
| Try different phrasings and keywords       | Rephrase your prompt using synonyms or alternative sentence structures.      |
| Adjust the level of detail and specificity | Add or remove information to fine-tune the output.                           |
| Test different prompt lengths              | Experiment with both shorter and longer prompts to find the optimal balance. |

## 6. Leverage Chain of Thought Prompting:

| Tactic                                                | Prompt Example                                                                                                                                                                                                                                          |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Encourage step-by-step reasoning                      | "Solve this problem step-by-step: John has 5 apples, he eats 2. How many apples does he have left? Step 1: John starts with 5 apples. Step 2: He eats 2 apples, so we need to subtract 2 from 5. Step 3: $5 - 2 = 3$ . Answer: John has 3 apples left." |
| Ask the model to explain its reasoning process        | "Explain your thought process in determining the sentiment of this movie review: 'The acting was superb, but the plot was predictable.'"                                                                                                                |
| Guide the model through a logical sequence of thought | "To classify this email as spam or not spam, consider the following: 1. Is the sender known? 2. Does the subject line contain suspicious keywords? 3. Is the email offering something too good to be true?"                                             |

## Temperature — Controls randomness / creativity

**Definition:** Temperature determines how random or deterministic the model's output is.

A low temperature (0–0.3) → model gives more focused, predictable answers

A high temperature (0.7–1.0+) → model gives more creative, varied, and surprising responses

### Analogy:

Think of "temperature" like spice level in food 🌶️ —

lower spice (0.2) = safe, consistent flavor

higher spice (1.0) = adventurous, surprising flavor

Example Prompt:

Suggest a name for a coffee shop.

| Temperature | Possible Output  |
|-------------|------------------|
| 0.2         | The Coffee Spot  |
| 0.7         | Brew Haven       |
| 1.0         | Caffeine Carouse |

Use low temperature for tasks like summarization, factual Q&A, or code generation.

Use high temperature for brainstorming, creative writing, or ad copy.

## Top-p (a.k.a. Nucleus Sampling) — Controls probability threshold

**Definition:** Top-p limits the range of possible next words the model can choose from — based on their cumulative probability. Instead of taking the top k most likely words (as in Top-k sampling), it takes the smallest set of words whose combined probability = p.

**Example:** If top-p = 0.9, the model considers only the most probable words that together make up 90% of the likelihood mass — ignoring the rare 10%.

### Analogy:

Imagine the model has a “bag” of possible next words.  
Top-p decides how deep into that bag it can reach:

Low top-p (0.3) → only the most predictable words

High top-p (0.9) → more diverse, riskier word choices

### Usage Tip:

Top-p and temperature both control randomness, but in different ways: Temperature spreads or sharpens all probabilities and Top-p cuts off the least likely words

You often keep one fixed and tune the other (e.g., top-p=1.0, adjust temperature).

## Max Tokens — Controls length of the output

### Definition:

Max tokens = the maximum number of tokens the model is allowed to generate in its response.

1 token ≈ 4 characters in English (roughly  $\frac{3}{4}$  of a word)

Example: “ChatGPT is amazing!” → 4 tokens

### Example:

If you set max\_tokens = 50, the model will stop after generating ~50 tokens, even if the answer feels incomplete.

### Why it matters:

Prevents runaway responses

Controls cost (since you’re billed per token)

Lets you design structured outputs (like short summaries or answers)

## Let's build a great prompt from scratch in the console

1. Task context

2. Tone context

3. Background data, documents, and images

4. Detailed task description & rules

5. Examples

6. Conversation history

7. Immediate task description or request

8. Thinking step by step / take a deep breath

9. Output formatting

10. Prefilled response (if any)

Prompting 101 | Code w/ Claude



Anthropic  
291K subscribers

Subscribe

👍 3.5K



🔗 Share

⬇️ Download

✂️ Clip



# 10-Step Framework for Effective Prompting

This framework outlines the key components for creating clear, structured, and high-quality prompts for any large language model (LLM). It can be applied across various use cases, from data analysis and reasoning to creative writing and customer support automation.

## 1. Task Context

Clearly define the model's role and what it is expected to do.

For example, specify that it is an AI assistant helping a claims adjuster review car accident report forms or a teaching assistant grading assignments.

Giving clear-cut instructions prevents the model from having to guess what the user wants and helps it focus on the correct domain.

## 2. Tone Context

Define the required behavior and communication style.

For instance, instruct the model to stay factual, confident, and concise.

If the model cannot understand the content or is not fully confident, it should not guess or mislead.

Setting tone guidelines ensures consistency, reliability, and safety in outputs.

## 3. Background Data / Documents / Images

Provide all the content the model needs to complete the task.

This content typically falls into two categories:

- **Dynamic Content:** The input that changes with each query (e.g., specific images, forms, or data to analyze).
- **Background Detail / Static Data:** Context that remains constant across different queries (e.g., form structure, reference documents, templates).

Providing this background information helps the model understand the context and structure of the task more effectively.

### **Structure and Organization:**

Language models perform best with clear organization. Use formatting tools to separate and label content:

- **Delimiters:** Use delimiters like XML tags (`<user_preferences>`) or Markdown to structure sections and help the model refer back to them later.

## **4. Detailed Task Description and Rules**

Instead of just stating the goal, provide detailed, step-by-step instructions for how the task should be performed and how the model should reason through it.

- **Ordering of Analysis:** The sequence in which the model analyzes information is very important.
- **Show its Work:** If you ask the model to “carefully examine each and every box,” it will often show its intermediate reasoning, which can be useful for transparency or debugging.

## **5. Incorporate Examples**

Examples (also known as *few-shot prompting*) significantly improve accuracy and reliability.

Providing examples of both correct and tricky cases helps the model understand desired outcomes.

By embedding examples, you enable the model to reference them later when handling similar situations.

## **6. Adding Conversation History**

If the model is used in an ongoing dialogue (e.g., a chatbot or assistant), include relevant conversation history in the prompt.

This gives the model context for the current exchange and ensures it can maintain continuity, tone, and understanding across turns.

## **7. Immediate Task Description or Request**

Reiterate the immediate task and key guidelines before the model responds.  
This reminder helps prevent hallucinations or off-topic responses.

You can also instruct the model to:

- Only respond if it is confident in its understanding.
- Refer back to the provided context or document whenever making factual claims.

## 8. Thinking Step-by-Step

Encourage the model to reason step-by-step.

Provide instructions that describe the order in which it should think, analyze, and conclude.

This “chain-of-thought” style prompting often leads to more accurate, logical, and transparent results — especially for complex reasoning or data analysis tasks.

When told to “carefully examine step by step,” models often show their reasoning process, which can be useful for understanding how the answer was derived.

## 9. Output Formatting

Specify exactly how the output should be presented, especially if it will be used for further processing or stored in a database.

Ask the model to wrap its final output in clear delimiters such as XML tags (e.g., `<final_verdict>`) or structured JSON format.

Defining the output structure helps ensure clarity, consistency, and ease of automation.

## 10. Pre-Filled Responses

To guide the model toward a specific structure right from the start, you can use **pre-filled responses**.

For instance, instruct the model to begin its output with a specific format such as an open XML tag or a square bracket.

This technique ensures the output starts in the right structure and adheres to formatting rules throughout.

## Let's Practice: Building a Vehicle Total Loss Prediction Prompt

We're going to create a prompt to predict vehicle total loss. The goal is to make the AI's decisions consistent, explainable, and data-driven. By following structured reasoning and standardized JSON outputs, we can:

- Reduce manual effort for claims adjusters
- Speed up claim decisions

- Improve transparency in how AI evaluates repair costs versus vehicle market value

Let's break this down step by step.

### Step 1: Task Context

Instruction: `You are an AI assistant specialized in insurance claims valuation.`

Explanation:

- This tells the AI what its role is.
- It focuses the model on insurance, vehicle evaluation, and claims reasoning.
- Think of it as setting the "hat" the AI is wearing.

### Step 2: Tone Context

Instruction: `Tone: factual, concise, conservative`

Explanation:

- Ensures responses are clear, safe, and professional.
- The AI should avoid guessing or exaggerating costs.
- "Conservative" means better to flag uncertainty than risk a wrong recommendation.

### Step 3: Background Data / Documents / Images

Instruction:

`Default total_loss_threshold = 0.70`

`Currency: INR`

Explanation:

- Sets static reference values for all claims.
- Threshold tells the AI when a repair is considered a total loss ( $\geq 70\%$  of market value).
- Currency ensures monetary values are interpreted correctly.

### Step 4: Detailed Task Description and Rules

Instruction: `If required numeric inputs are missing or invalid, return verdict: "INSUFFICIENT_INFO" and list the missing/invalid fields in "explanations".`

Dynamic Example Input:

`<dynamic>`

```
{
  "claim_id": "C20001",
  "vehicle": {
    "make": "Honda",
    "model": "City",
    "year": 2019,
    "variant": "ZX",
    "odometer_km": 42000
  },
  "repair_estimate": {
    "parts_total": 45000,
    "labor_charges": 6750,
    "other_charges": 2500
  },
  "market_value": 600000,
  "salvage_value": 50000,
  "total_loss_threshold": 0.70
}
</dynamic>
```

Explanation:

- Dynamic inputs change per claim.
- The AI uses this data to calculate repair cost, ratio, and determine total loss.

### Step 5: Incorporate Examples (Few-Shot Prompting)

Example 1 Input:

```
{
  "claim_id": "E1",
  "vehicle": {"make": "Hyundai", "model": "i20", "year": 2018},
  "repair_estimate": {"parts_total": 20000, "labor_charges": 3000},
  "market_value": 250000
}
```

Example 1 Output:

```
{
  "claim_id": "E1",
  "verdict": "REPAIRABLE",
  "repair_cost": 23000,
  "market_value": 250000,
  "market_value_source": null,
  "repair_to_market_ratio": 0.092,
  "total_loss_threshold": 0.7,
  "total_loss": false,
  "salvage_value": null,
  "payable_amount": 23000,
}
```

```
"confidence":0.95,
"explanations":["repair_cost parts+labor","ratio 0.092 < 0.7"]
}
```

Example 2 Input:

```
{
  "claim_id":"E2",
  "vehicle":{"make":"Maruti","model":"Swift","year":2016},
  "repair_estimate":{"parts_total":350000,"labor_charges":30000},
  "market_value": 500000,
  "salvage_value": 20000
}
```

Example 2 Output:

```
{
  "claim_id":"E2",
  "verdict":"TOTAL_LOSS",
  "repair_cost":390000,
  "market_value":500000,
  "market_value_source":"estimated_from_valuation_table_2025",
  "repair_to_market_ratio":0.78,
  "total_loss_threshold":0.7,
  "total_loss":true,
  "salvage_value":20000,
  "payable_amount":370000,
  "confidence":0.88,
  "explanations":["market value estimated","ratio 0.78 >= 0.7"]
}
```

Explanation:

- Examples teach the AI expected inputs, outputs, and reasoning.
- Few-shot prompting improves accuracy and reliability.

### Step 6: Adding Conversation History

- Not included here because this is a single-turn
- In multi-turn chatbots, include past interactions to maintain context.

### Step 7: Immediate Task Description / Request

Instruction: Only return the final JSON object exactly as specified.

Round all monetary values to the nearest rupee.

Explanation:

- Focuses the AI on producing structured, parseable output for downstream applications.
- Avoids extra text or formatting mistakes.

### Step 8: Thinking Step-by-Step

Instruction: If `repair_cost` is extremely high vs vehicle age or ratio within  $\pm 0.05$  of threshold, include "needs\_manual\_review" in explanations.

Step-by-Step Logic for AI:

1. Validate numeric inputs; return `INSUFFICIENT_INFO` if invalid.
2. Compute `repair_cost = parts_total + labor_charges + other_charges`.
3. Use provided `market_value`; otherwise estimate and store `market_value_source`.
4. Compute `repair_to_market_ratio = repair_cost / market_value`.
5. Determine `total_loss`: ratio  $\geq$  threshold  $\rightarrow$  `TOTAL_LOSS`; else `REPAIRABLE`.
6. Compute `payable_amount = max(0, repair_cost - salvage_value)` if salvage exists.
7. Assign confidence score; reduce if `market_value` estimated or ratio borderline.
8. Include 1–3 short explanation bullets.
9. Return only JSON matching schema.
10. For low confidence ( $<0.8$ ) or borderline ratio, include `<analysis>` block with calculations.

### Step 9: Output Formatting

- Only return JSON objects.
- Round all monetary values to the nearest rupee.
- Ensures consistency for downstream systems.

### Step 10: Pre-Filled Responses

- Provide skeleton JSON to guide the model:

{

```
"claim_id": "",
"verdict": "",
"repair_cost": 0,
"market_value": 0,
"market_value_source": null,
"repair_to_market_ratio": 0.0,
"total_loss_threshold": 0.7,
"total_loss": false,
"salvage_value": null,
"payable_amount": null,
"confidence": 0.0,
"explanations": []
}
```

Explanation:

- Helps the AI always follow the expected structure.
- Minimizes errors and ensures downstream automation works smoothly.

## The Complete Prompt for the AI

You are an AI assistant specialized in insurance claims valuation.

Tone: factual, concise, conservative.

Do not guess. If required numeric inputs are missing or invalid, return verdict: "INSUFFICIENT\_INFO" and list the missing/invalid fields in "explanations".

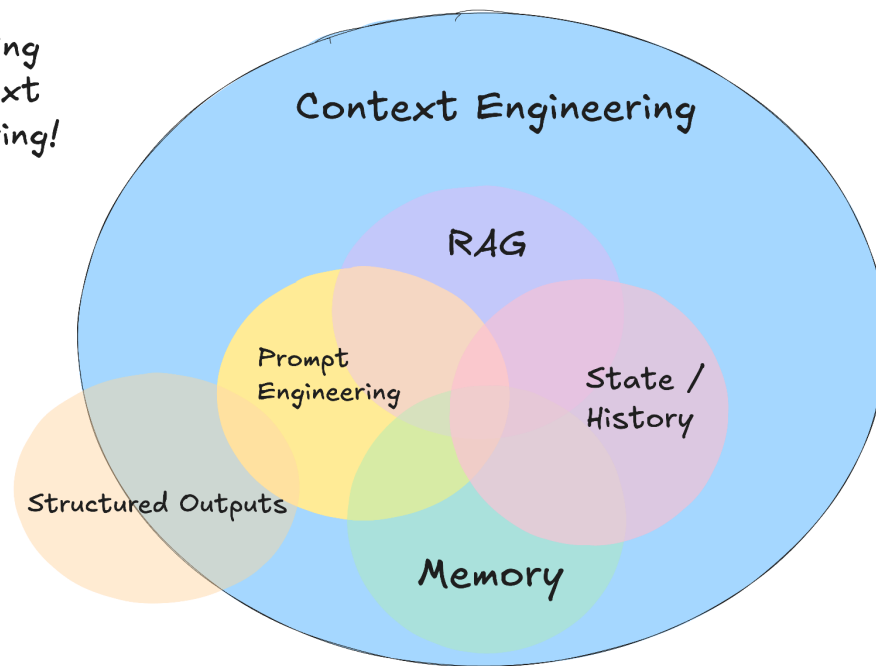
Default total\_loss\_threshold = 0.70. Currency: INR. Round all monetary values to the nearest rupee.

Only return the final JSON object exactly as specified.

If repair\_cost is extremely high vs vehicle age or ratio within  $\pm 0.05$  of threshold, include "needs\_manual\_review" in explanations.

## CONTEXT ENGINEERING

Everything  
is Context  
Engineering!



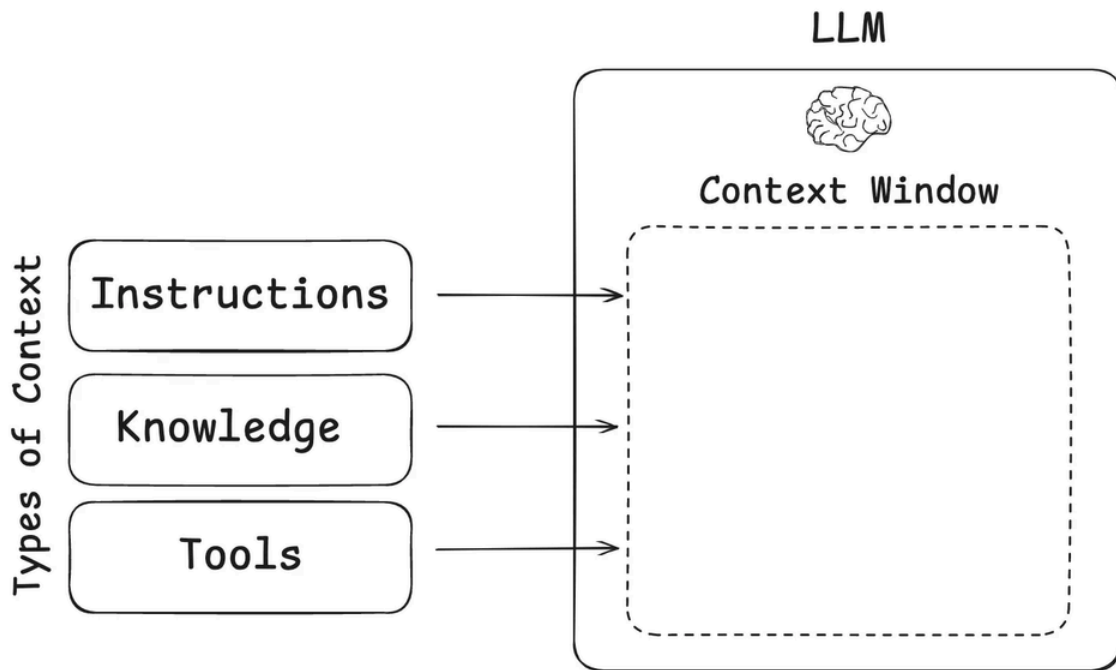
### **What is Context?**

Context refers to the set of tokens included when sampling from a large-language model (LLM)

### **What is Context Engineering?**

It is the natural progression of prompt engineering. Context engineering refers to the set of strategies for curating and maintaining the optimal set of tokens (information) during LLM inference, including all the other information that may land there outside of the prompts

LLMs are like a new kind of operating system. The LLM is like the CPU and its context window is like the RAM, serving as the model's working memory. Just like RAM, the LLM context window has limited capacity to handle various sources of context. And just as an operating system curates what fits into a CPU's RAM, we can think about "context engineering" playing a similar role



**What are the types of context that we need to manage when building LLM applications?**

Context engineering as an umbrella that applies across a few different context types:

**Instructions** – prompts, memories, few-shot examples, tool descriptions, etc

**Knowledge** – facts, memories, etc

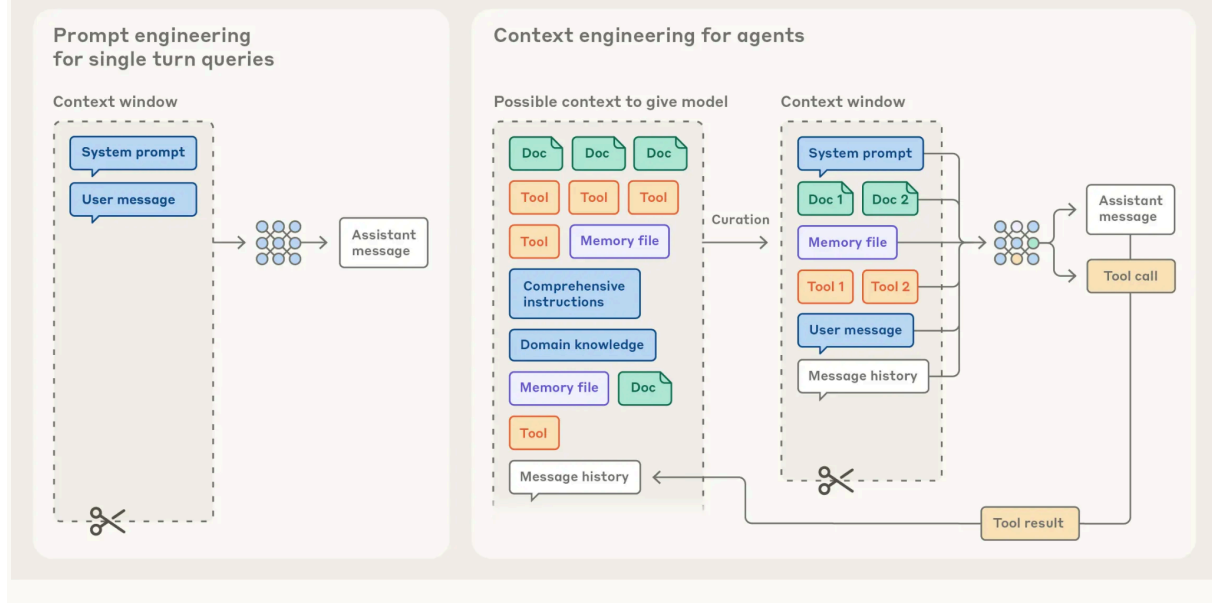
**Tools** – feedback from tool calls

### **Context engineering vs. prompt engineering**

The primary focus of prompt engineering is how to write effective prompts, particularly system prompts. However, as we move towards engineering more capable agents that operate over multiple turns of inference and longer time horizons, we need strategies for managing the entire context state (system instructions, tools, Model Context Protocol (MCP), external data, message history, etc).

An agent running in a loop generates more and more data that could be relevant for the next turn of inference, and this information must be cyclically refined. Context engineering is the art and science of curating what will go into the limited context window from that constantly evolving universe of possible information.

# Prompt engineering vs. context engineering



## Why is context engineering important to building capable agents?

Despite their speed and ability to manage larger and larger volumes of data, it is observed that LLMs, like humans, lose focus or experience confusion at a certain point. Studies on needle-in-a-haystack style benchmarking have uncovered the concept of **context rot**: as the number of tokens in the context window increases, the model's ability to accurately recall information from that context decreases.

While some models exhibit more gentle degradation than others, this characteristic emerges across all models. **Context, therefore, must be treated as a finite resource with diminishing marginal returns.** Like humans, who have limited working memory capacity, LLMs have an **"attention budget"** that they draw on when parsing large volumes of context. Every new token introduced depletes this budget by some amount, increasing the need to carefully curate the tokens available to the LLM.

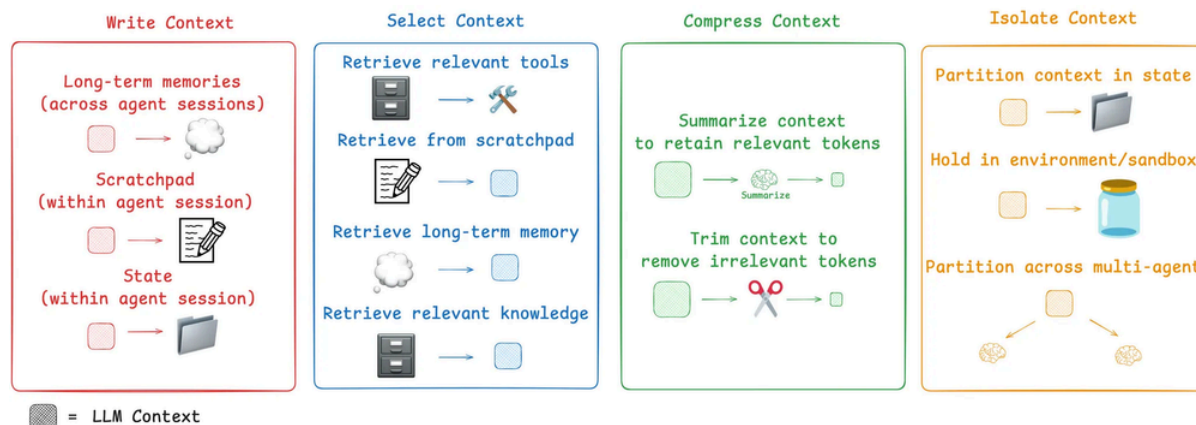
This attention scarcity stems from architectural constraints of LLMs. **LLMs are based on the transformer architecture**, which enables every token to attend to every other token across the entire context. This results in  **$n^2$  pairwise relationships for  $n$  tokens.**

As its context length increases, a model's ability to capture these pairwise relationships gets stretched thin, creating a natural tension between context size and attention focus. Additionally, models develop their attention patterns from training data distributions where shorter sequences are typically more common than longer ones. This means models have less experience with, and fewer specialized parameters for, context-wide dependencies.

Techniques like **position encoding interpolation** allow models to handle longer sequences by adapting them to the originally trained smaller context, though with some degradation in token position understanding. These factors create a performance gradient rather than a hard cliff: models remain highly capable at longer contexts but may show reduced precision for information retrieval and long-range reasoning compared to their performance on shorter contexts.

These realities mean that thoughtful context engineering is essential for building capable agents.

## Common Strategies for agent context engineering



**1. Write Context:** Writing context means saving it outside the context window to help an agent perform a task. Agents take note via a “**scratchpad**” to persist information while it is performing a task. The idea is to save information outside of the context window so that it’s available to the agent. Scratchpads can be implemented in a few different ways. They can be a tool call that simply writes to a file. They can also be a field in a runtime state object that persists during the session. In either case, scratchpads let agents save useful information to help them accomplish a task.

**Memories:** Scratchpads help agents solve a task within a given session (or thread), but sometimes agents benefit from remembering things across many sessions! Reflexion introduced the idea of reflection following each agent turn and re-using these self-generated memories. Generative Agents created memories synthesized periodically from collections of past agent feedback.

**2. Select Context:** Selecting context means pulling it into the context window to help an agent perform a task.

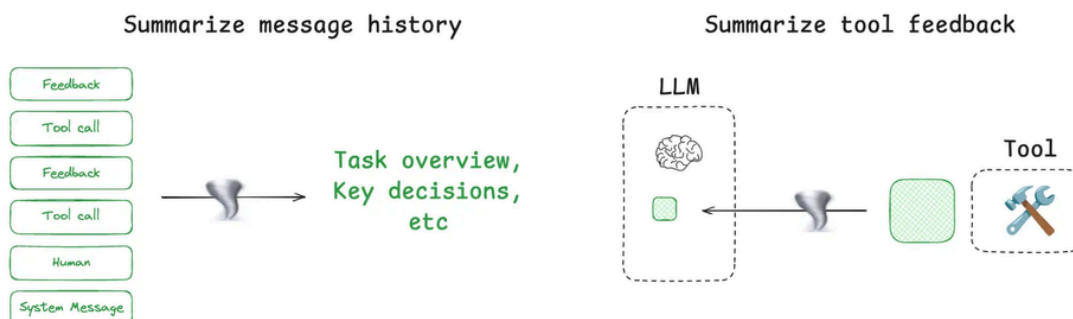
**Scratchpad:** The mechanism for selecting context from a scratchpad depends upon how the scratchpad is implemented. If it’s a tool, then an agent can simply read it by making a tool call. If it’s part of the agent’s runtime state, then the developer can choose what parts of state to expose to an agent each step. This provides a fine-grained level of control for exposing scratchpad context to the LLM at later turns.

**Memories:** If agents have the ability to save memories, they also need the ability to select memories relevant to the task they are performing. This can be useful for a few reasons. Agents might select few-shot examples (episodic memories) for examples of desired behavior, instructions (procedural memories) to steer behavior, or facts (semantic memories) for task-relevant context.

| Memory Type | What is Stored | Human Example              | Agent Example       |
|-------------|----------------|----------------------------|---------------------|
| Semantic    | Facts          | Things I learned in school | Facts about a user  |
| Episodic    | Experiences    | Things I did               | Past agent actions  |
| Procedural  | Instructions   | Instincts or motor skills  | Agent system prompt |

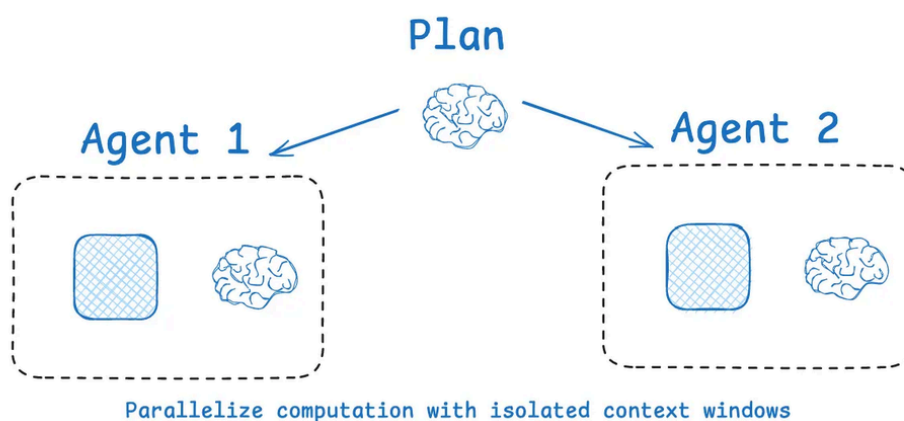
**3. Compressing Context:** Compressing context involves retaining only the tokens required to perform a task.

**Context Summarization:** Agent interactions can span hundreds of turns and use token-heavy tool calls. Summarization is one common way to manage these challenges.



**4. Isolating Context:** Isolating context involves splitting it up to help an agent perform a task.

**Multi-agent:** One of the most popular ways to isolate context is to split it across sub-agents. A motivation for the OpenAI Swarm library was separation of concerns, where a team of agents can handle specific sub-tasks. Each agent has a specific set of tools, instructions, and its own context window.



## Example Use Case:

Let's say we're designing an **AI Interview Coach** that helps users practice product management interviews by asking them realistic questions, evaluating their answers, and giving feedback.

We'll now explain how to **write, select, compress, and isolate context** for this use case.

### 1. Write Context

**Definition:** Writing context means explicitly defining the **role, task, goals, constraints, and output format** inside the context window — so the agent knows exactly *who it is* and *what to do*.

#### Implementation:

When the session starts, we “write” the key context for the LLM:

[ROLE]

You are an AI Interview Coach specializing in Product Management interviews.

[TASK]

Your goal is to simulate an interviewer by asking one question at a time, evaluating the candidate's response, and giving short, actionable feedback.

[BEHAVIOR]

- Be encouraging but honest.
- Avoid generic praise; focus on concrete improvement points.
- Keep each feedback under 80 words.

[OUTPUT FORMAT]

Return your responses in the following JSON format:

```
{  
  "question": "...",  
  "evaluation": "...",  
  "feedback": "..."  
}
```



#### Why this matters:

This “written context” gives the model full clarity on its **identity, mission, and response structure**, without needing prior sessions or memories.

### 2. Select Context

**Definition:** Selecting context means choosing **what past or external information** to reintroduce into the context window during each new turn.

**In Our Example:** As the user interacts with the AI Coach across multiple turns (say 10–15 questions), we don't want to overload the context window with all prior Q&A. So we selectively **pull only what's relevant**:

- The **latest user answer**
- The **previous question**
- A **short summary** of feedback so far

**Example Implementation:**

When the user answers question 5, the orchestrator script injects:

[SELECTED CONTEXT]

Previous question: "How do you prioritize features in a product roadmap?"

User's answer: "I use RICE scoring to align with business goals."

Last feedback summary: "Good framework usage, try adding more customer empathy."

[NEW INSTRUCTION]

Ask the next question based on their previous answers to maintain progression.

 **Why this matters:**

The agent doesn't need to see the entire transcript — just the context required for continuity and personalization.

### 3. Compressing Context

**Definition:** Compressing context means **shortening** or **summarizing** what's included in the context window so the model stays within token limits while retaining meaning.

**In Our Example:**

After 10–15 questions, the agent can summarize the interaction history:

[COMPRESSED CONTEXT]

Summary so far:

- Candidate strong in frameworks (RICE, Kano)
- Needs to improve in product metrics articulation
- Good storytelling but lacks data-driven validation

Instead of storing all prior Q&A (thousands of tokens), we keep a **summary snapshot**.

This can be done by:

- Using LLM-based summarization tools

- Periodic compression after every few turns



#### Why this matters:

Keeps the model efficient and focused on the user's learning trajectory instead of wasting tokens on old details.

## 4. Isolating Context

**Definition:** Isolating context means **modularizing** it — splitting the full context into different parts or even separate agents to handle sub-tasks independently.

#### In Our Example:

We can isolate context into **three sub-agents**, each with their own local context:

| Sub-Agent               | Role                                     | Context Example                                              |
|-------------------------|------------------------------------------|--------------------------------------------------------------|
| <b>Question Agent</b>   | Generates the next PM interview question | Focuses only on question difficulty level and topic rotation |
| <b>Evaluation Agent</b> | Evaluates user's answer                  | Uses evaluation rubric context                               |
| <b>Feedback Agent</b>   | Provides improvement advice              | Accesses summarized user performance context                 |

Each sub-agent has **its own context window** tailored to its function.

A central orchestrator coordinates between them — passing summaries, not full transcripts.



#### Why this matters:

Prevents context clutter, improves reasoning quality, and mirrors real-world modular architectures (like OpenAI Swarm or CrewAI).

# Retrieval Augmented Generation

One of the most powerful applications enabled by LLMs is sophisticated question-answering (Q&A) chatbots. These are applications that can answer questions about specific source information. These applications use a technique known as **Retrieval Augmented Generation, or RAG**.

## Let's understand it with the help of examples:

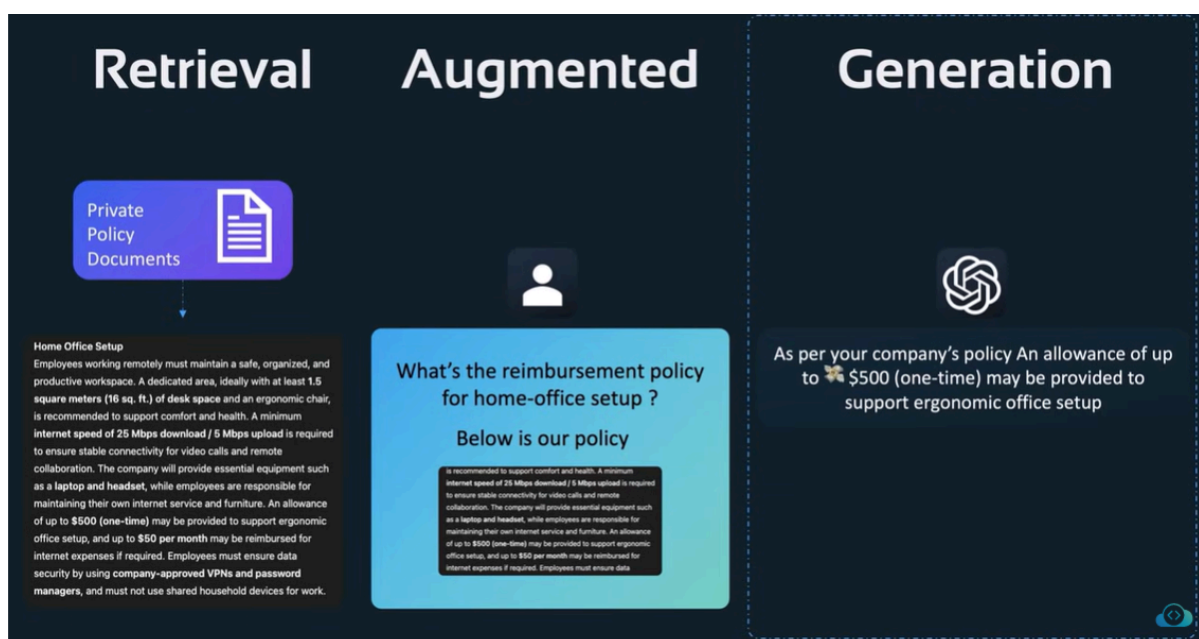
When an LLM, such as ChatGPT, is asked a question about private data—like a company's reimbursement policy—it cannot access that private document. In this situation, the LLM lacks the necessary context and will typically hallucinate (provide an incorrect or generic answer).

In such a scenario we will give the entire office policy document/content to ChatGPT and then it will be able to refer to the company's reimbursement policy and answer the asked question.

## The Three Components of RAG

The acronym RAG breaks down into three fundamental processes that happen when a query is submitted:

1. Retrieval (R): This is the part where the system looks up internal documents (or external knowledge bases) and retrieves the relevant information.
2. Augmenting (A): This involves using the retrieved information to improve or augment the user's prompt before it is sent to the LLM.
3. Generation (G): This is the final step where the LLM generates a response based on the context provided in the augmented prompt.



## RAG VS fine tuning VS Prompt Engineering

|                    | Purpose                                                                                                                                                                | Type of Information                                                   | Mechanism                                                                                                                                                                                                       | Example Use Case                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RAG                | To get the most accurate responses by retrieving the most relevant, current information dynamically at query time                                                      | Dynamic factual information, such as internal company policies        | The system automatically locates and retrieves relevant external documents (Retrieval), adds this information to the user's question (Augmentation), and then sends the complete prompt to the LLM (Generation) | An employee asks, "What's the reimbursement policy for home office setup?". The RAG system searches the private company policy documents, retrieves the exact section about home office reimbursement, and feeds that section to the LLM to generate an accurate answer.                                                                                                                      |
| Fine Tuning        | To impart stable, unchanging patterns like communication style, tone, and language                                                                                     | Communication style, tone, specific language/jargon                   | Providing a model with hundreds of sample questions and sample answers and having it respond in that specific way all the time. This often involves retraining the model.                                       | Building a "policy co-pilot" that has a specific voice and style, such as the authoritative and distinctly Scottish accent and candor of the CEO. The fine-tuning model uses the CEO's past speeches, emails, and writings to learn to respond in the same tone and language.                                                                                                                 |
| Prompt Engineering | To apply restrictions and security rules, defining what the chatbot must reveal and what it must not. It acts as a rule book to keep the chatbot safe and professional | Instructions for behavioral control, limitations, and security rules. | Providing strict instructions to the LLM to control its behavior based on user requests                                                                                                                         | Ensuring the chatbot does not answer questions that are "off limits," such as performance review appeals or salary discussions. For sensitive topics, the prompt engineering rule might instruct the LLM to politely redirect users to HR instead of providing answers in the chat. The instruction could also be to "never to reveal personal employee information or confidential details". |

When we perform the Retrieval in RAG we need searches and then most important type of searches are keyword search and semantic search

### Keyword search

It is a traditional approach to finding information in documents based on exact word matches. It is described as a very popular technique used by many search platforms.

In a system using keyword search, if a user's query does not contain the specific term found in a document, the search is likely to fail.

### Mechanism and Techniques

Keyword search functions by ranking documents based on the occurrence and frequency of the query terms within them.

**1. Identification and Ranking:** The process goes through all documents, identifies the keywords from the user's question (e.g., "reimbursement" and "home office"), and then ranks the documents based on their frequency.

**2. Counting Occurrences:** The system counts the occurrences of each keyword in all documents. By combining the counts for all keywords, the system attempts to identify the document that has the maximum occurrences of those terms, thereby selecting the relevant document.

Two of the most popular proven approaches available for implementing keyword search are TF (Term Frequency) and BM25 (Best Match 25).

- **TF-IDF Example:** The TF-IDF (Term Frequency-Inverse Document Frequency) method uses an analyzer to calculate word scores, which show the importance of each word in a sentence.

- **BM25 Example:** The BM25 algorithm is stricter in assigning scores and assigns a score to the most important and unique words in sentences.

### Limitations of Keyword Search

The fundamental limitation of keyword search is that it can only return content that exactly matches the keywords that are being searched for, leading to a low chance of getting accurate information every time.

- **Lack of Semantic Understanding:** If the user uses a synonym instead of the exact keyword, the search fails. For example, if a user searches for "allowance" instead of "reimbursement," or "work from home" instead of "home office," the search is often unable to find matches because those combinations of keywords are not present in the documents.

- **Poor Results for Context:** A pure keyword-based search is less likely to yield the right results. A demonstration showed that searching for a query like "distributed workforce policies" resulted in most document scores being zero because the exact phrase did not exist in the documents.

This limitation of only matching exact words is why RAG systems use semantic search and vector search, which search documents based on the meaning of the words, rather than just the word overlap, making RAG more powerful than traditional keyword search

### Semantic Search

Semantic search is a technique that searches documents based on the meaning of words.

#### Core Concept

The primary goal of semantic search is to have a much higher chance of locating the correct documents based on the user's input, even if the user does not use the exact keywords present in the documents.

For example, if a user searches for "allowance," semantic search can successfully find documents discussing "reimbursements," or anything else with a similar meaning. Similarly, searching for "work from home" can retrieve documents related to "home office" or "remote work" policies.

## The Mechanism: Embeddings and Vectors

The "magic" behind semantic search happens through the use of embeddings.

**1. Conversion to Vectors:** Both your search query and all of the documents are converted into mathematical vectors. A vector is an array of numbers that represents the text. This conversion is done by an embedding model.

**2. Representing Meaning:** These vectors act as coordinates in a high-dimensional space. The numbers in the array capture different aspects of the text's meaning (e.g., how animal-related or how workplace-related a sentence is).

**3. Plotting:** Documents or sentences with similar meanings end up close together in this high-dimensional space. For instance, "dogs" might be added to a section of the graph that represents animals, and "pets" would fall into the same category, while "remote" would not.

**4. Similarity Measurement:** When you search, the system finds the closest matches based on the meaning, not just word overlap. You can measure how similar two pieces of text are by calculating the distance between their vectors. The shorter the distance, the more similar the meaning. For example, the vectors for "reimbursement" and "allowance" would be close together, even though they are different words.

### Calculating Similarity Using the Dot Product

Since these vectors often exist in high dimensions (e.g., the all-MiniLM-L6V2 model uses 384 dimensions), the similarity cannot be visualized easily. Instead, a mathematical approach called the dot product is used.

- The dot product acts as a "mathematical ruler" that can measure the distance between two points (vectors) in any number of dimensions.
- The process involves multiplying the corresponding values in the vectors, adding them together, and then applying a normalization process.
- The final calculation returns a similarity score between 0 and 1. A pair of texts with a score closest to 1 are similar, and those away from 1 are dissimilar.
- For example, when similarity scores were calculated between test sentences:
  - "Dogs versus pets" showed 73.3% similarity.
  - "Dogs versus remote" showed only 36.2% similarity.

This ability to find semantically similar content, even when exact words do not match, is what makes RAG systems so powerful compared to traditional keyword search.

## Embedding models

Embedding models are fundamental components of modern systems like Retrieval Augmented Generation (RAG) and are essential for enabling semantic search.

Here is a comprehensive explanation of embedding models :

**Core Function and Definition:** An embedding model is a type of machine learning model designed to take text and convert it into numbers that represent meaning.

- **Output:** The output of an embedding model is an array of numbers, known as a vector.
- **Vector Space:** The model maps sentences and paragraphs into a high-dimensional dense vector space.

In this space, documents or text segments that have similar meanings end up being close together.

- **Capturing Meaning:** When the model processes a sentence (e.g., "dogs are allowed in the office"), it does not just look at the words; it thinks about the actual meaning (is it about animals, workplace policies, or permissions?). The resulting list of numbers (the vector) captures these different aspects of meaning.

Role in Semantic Search and Similarity

Embedding models are the "magic" that makes semantic search possible, allowing the system to search based on the meaning of words, not just exact word matches (keyword search limitations).

- **Measuring Similarity:** The similarity between two pieces of text is measured by calculating the distance between their corresponding vectors. The closer the vectors are, the more similar the meaning they convey. For example, the vectors for "reimbursement" and "allowance" would be close together because they share similar meaning.

- **High-Dimensional Space:** Since these vectors often exist in high dimensions (sometimes hundreds, like 384 dimensions), the similarity is calculated mathematically using techniques like the dot product, which acts as a "mathematical ruler" to measure distance. The dot product process involves multiplying, adding, and normalizing the vector values. Pairs of sentences with a final score closest to 1 are considered similar.

Example: The All Mini LM L6V2 Model

## Vector Database

A vector database (Vector DB) is a specialized type of database designed to efficiently store, manage, and search mathematical vectors (or embeddings). These databases are critical components of the Retrieval Augmented Generation (RAG) pipeline because they allow systems to retrieve relevant results instantly and scalably.

### The Purpose of a Vector Database

Vector databases solve the problem of scale that arises when trying to perform semantic search across a large collection of documents.

**The Problem Solved:** Imagine a policy co-pilot system that needs to search through hundreds or thousands of large policy documents, such as 500 documents, each with multiple sections.

If the system were to compare a user's query embedding with every single stored embedding (the "naive way"), it would require a massive number of calculations for every query. For instance, 500 documents, each with 384 dimensions, would require 192,000 calculations per query, which is too slow for real-world application.

**The Solution:** The vector database acts like a "smart librarian who knows exactly where to look". It efficiently uses resources and is scalable, enabling instant retrieval of relevant results.

## Key Mechanisms and Concepts

### 1. Smart Indexing Algorithms

Instead of comparing every vector individually, vector databases pre-organize the vectors using smart indexing algorithms.

- **Neighborhoods:** This indexing process groups the vectors into "neighborhoods" based on their semantic similarity. For example, all animal policies are grouped together, all health benefits are grouped, and all remote work policies are grouped.
- **Targeted Search:** When a query (e.g., about bringing a dog to work) comes in, the system goes directly to the relevant neighborhood (e.g., the animal policies neighborhood) and only searches there, rather than searching the entire space.

### 2. Indexing Algorithms Used

- **HNSW (Hierarchical Navigable Small World):** This is the most widely used algorithm. It creates a graph structure where each vector is connected to its most similar neighbors. When searching, it starts from a random point and follows the connections to find the closest matches. HNSW is fast and accurate, which is why most vector databases use it by default.
- **IVF (Inverted File) and LSH (Locally Sensitive Hashing):** These are other examples of indexing algorithms.

## Popular Vector Database Implementations

- **Chroma:** This is an open-source and Python-friendly database, making it ideal for learning and experimentation. It can be installed locally, is free, and is excellent for students and small projects. Chroma is non-persistent by default (data is stored in memory and lost when the program stops), but it can be made persistent by specifying a path to store the database files.
- **Pinecone:** This is a managed service designed for large companies in production. The service provider handles all the infrastructure, and users pay per use.

## Chunking

- **Chunking** is a fundamental prerequisite step in the Retrieval Augmented Generation (RAG) pipeline. It is the process of breaking down large policy documents, such as a 50-page employee handbook, into smaller, focused pieces called chunks.

Chunking, along with creating embeddings and storing them in a vector database, must be done before a user starts asking questions because these processes are time-consuming.

### Importance and Purpose of Chunking

The primary importance of chunking is solving the precision problem that arises when handling large documents.

**Without chunking (The Precision Problem):** If a large document (like a 50-page employee handbook) is added to the vector database as a single entry, when a user queries for specific

information (e.g., "What's the remote work policy?"), they would receive the entire 50-page handbook. This is not helpful and results in users being overwhelmed with irrelevant information because they have to search through everything to find what they actually need.

**With Chunking:** Chunking allows the system to handle large documents while still providing precise, relevant answers. When a user asks about remote work, they get back just the specific policy sections that are relevant, which provides clear, focused answers. Instead of searching through entire documents, the RAG system searches through these focused chunks that contain exactly what the user is looking for.

## Types and Strategies of Chunking

### 1. Fixed Size Chunks with Overlap

This is a simple and reliable strategy for most use cases:

- **Fixed Size:** Documents are split into pieces of a predetermined size, often between 200 to 500 characters, or simply 500 characters per chunk. This makes the implementation easy.
- **Overlap:** A critical addition is the overlap, typically 50 to 100 characters (or 50 characters), added between chunks.
  - **The necessity of Overlap:** Fixed-size splitting risks breaking a sentence right in the middle (e.g., separating "dogs are allowed" from "on Fridays"), which breaks the meaning and makes it hard for the system to understand the complete information. The overlap ensures that the important context is preserved in both chunks.

### 2. Content-Based Chunking

- **Sentence-Based Chunking:** Every sentence is split into a separate chunk.
- **Paragraph-Based Chunking:** Each paragraph becomes a single chunk.

### 3. Advanced Methods

More advanced methods include semantic chunking and agentic chunking.

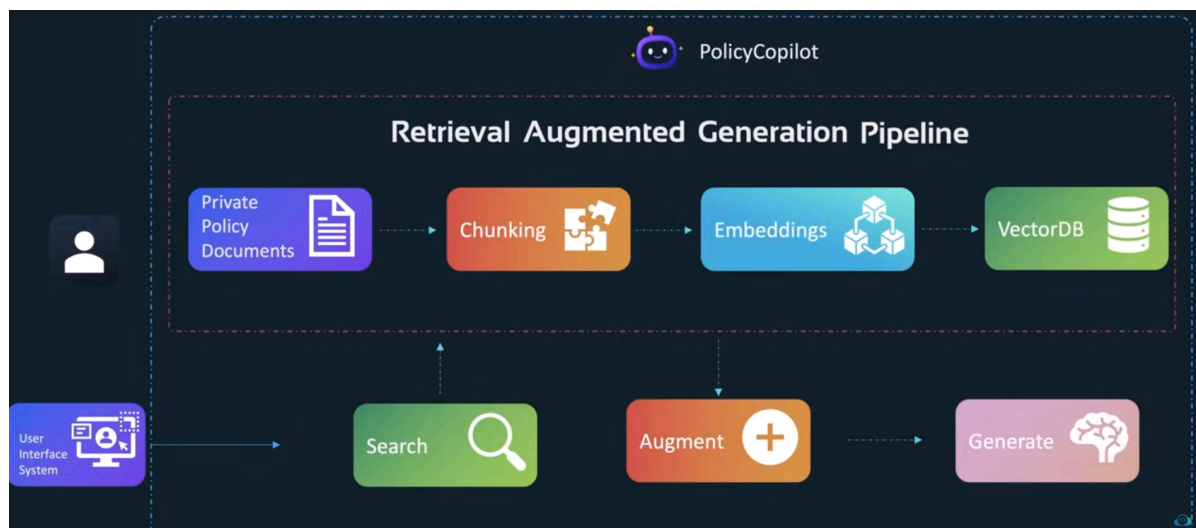
## The Challenge of Balance

While chunking sounds simple, it is tricky, and the main challenge is finding the right balance between context and precision.

**The RAG pipeline** is a structured process that enables Large Language Models (LLMs) to generate accurate responses by retrieving information dynamically from external knowledge sources. The pipeline can be understood in two main phases: the initial data ingestion/preparation pipeline, and the query-time execution flow (RAG).

### Phase 1: The Data Preparation Pipeline (Ingestion)

The steps involved in loading, chunking, and storing documents must be done before a user asks a question, as these processes are time-consuming.



**The simple RAG pipeline for data preparation involves the following steps:**

**1. Document Chunking:** Large policy documents (such as a 50-page employee handbook) are broken down into smaller, focused pieces called **chunks**.

- **Purpose:** Chunking solves the precision problem, ensuring that when a user searches, they receive only specific relevant sections, rather than the entire document.

- **Strategy:** A typical approach uses fixed size chunks (e.g., 500 characters) with an overlap (e.g., 50 characters) to ensure context is preserved if a split occurs mid-sentence.

**2. Embedding Generation:** The text chunks are converted into numerical representations known as vectors or embeddings.

- This conversion uses an embedding model (such as OpenAI's embedding models or the all mini LM L6V2 model). The vectors represent the semantic meaning of the text.

**3. Storage in a Vector Database (Vector DB):** The resulting vectors (embeddings) are loaded into a Vector DB (like ChromaDB or Pine Cone).

- The Vector DB uses indexing algorithms, such as Hierarchical Navigable Small World (HNSW), to pre-organize these vectors into "neighborhoods" (e.g., all remote work policies are grouped together). This allows for instant and efficient retrieval during the search phase.

## Phase 2: The Query-Time Execution Flow (R-A-G)

When a user asks a question (e.g., "What's the reimbursement policy for home office setup?"), the RAG system executes the three core components: Retrieval, Augmentation, and Generation.

### 1. Retrieval (R)

This is the process of retrieving the most relevant information.

- The user's query is first converted into a **vector/embedding**.
- The system performs a **semantic search** against the Vector DB, looking for stored document chunks whose vectors are mathematically closest to the query vector. The similarity is often calculated using the dot product.
- The system returns only the most relevant text chunks. This is effective because it finds semantically similar content even if the exact keywords are not used.

### 2. Augmentation (A)

This step involves improving or augmenting the original prompt with the retrieved information.

- The retrieved chunks (the context) are combined with the user's original question.

- A context-augmented prompt is created, which explicitly instructs the LLM to refer to the provided policy information. This ensures the LLM generates a factual, accurate response based on the external data.

### **3. Generation (G)**

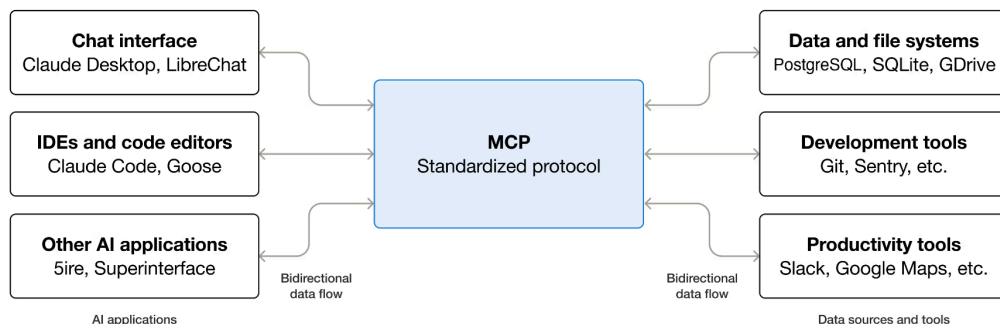
This is the final step where the LLM produces the answer.

- The LLM receives the augmented prompt containing both the question and the context.
- The LLM generates the final response based on the retrieved information.

## What is the Model Context Protocol (MCP)?

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems. Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.

Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.



## Why does MCP Exist?

Traditional LLMs operate within a fixed context window — they only “know” what’s inside the current prompt.

However, in real-world applications, models need access to:

- Private company data (e.g., customer tickets, sales reports)
- APIs and tools (e.g., Jira, Slack, Notion)
- Dynamic or structured context (e.g., user preferences, codebases)

RAG (Retrieval-Augmented Generation) and Plugins/APIs tried to solve this, but they each have limitations:

- RAG: only provides text chunks; limited interactivity
- Plugins: difficult to standardize, manage security, and reuse
- Ad-hoc APIs: brittle integrations and lack of visibility

Hence, MCP (Model Context Protocol) was introduced by Anthropic to standardize how models communicate with tools and context providers safely and modularly.

## What can MCP enable?

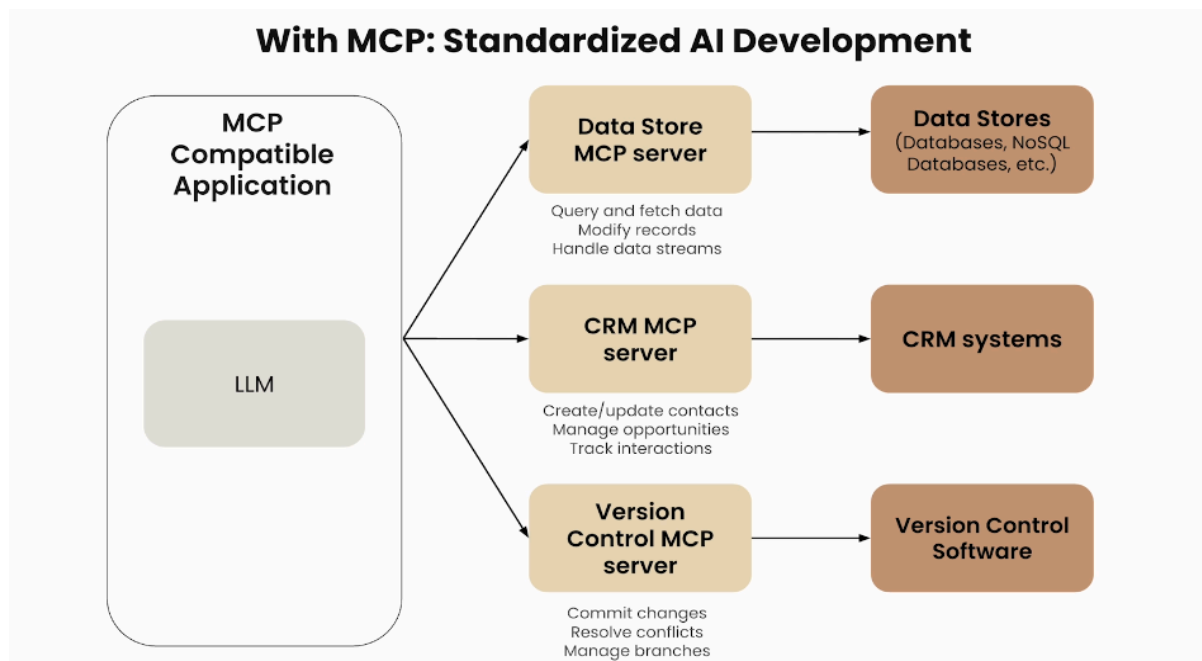
- Agents can access your Google Calendar and Notion, acting as a more personalized AI assistant.
- Claude Code can generate an entire web app using a Figma design.
- Enterprise chatbots can connect to multiple databases across an organization, empowering users to analyze data using chat.

- AI models can create 3D designs on Blender and print them out using a 3D printer.

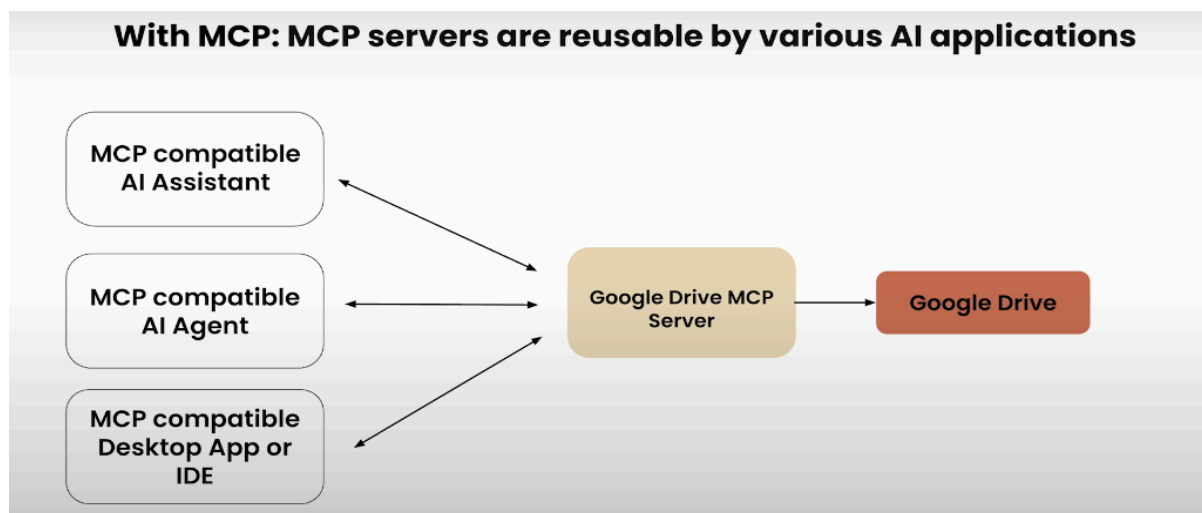
## Why does MCP matter?

Depending on where you sit in the ecosystem, MCP can have a range of benefits.

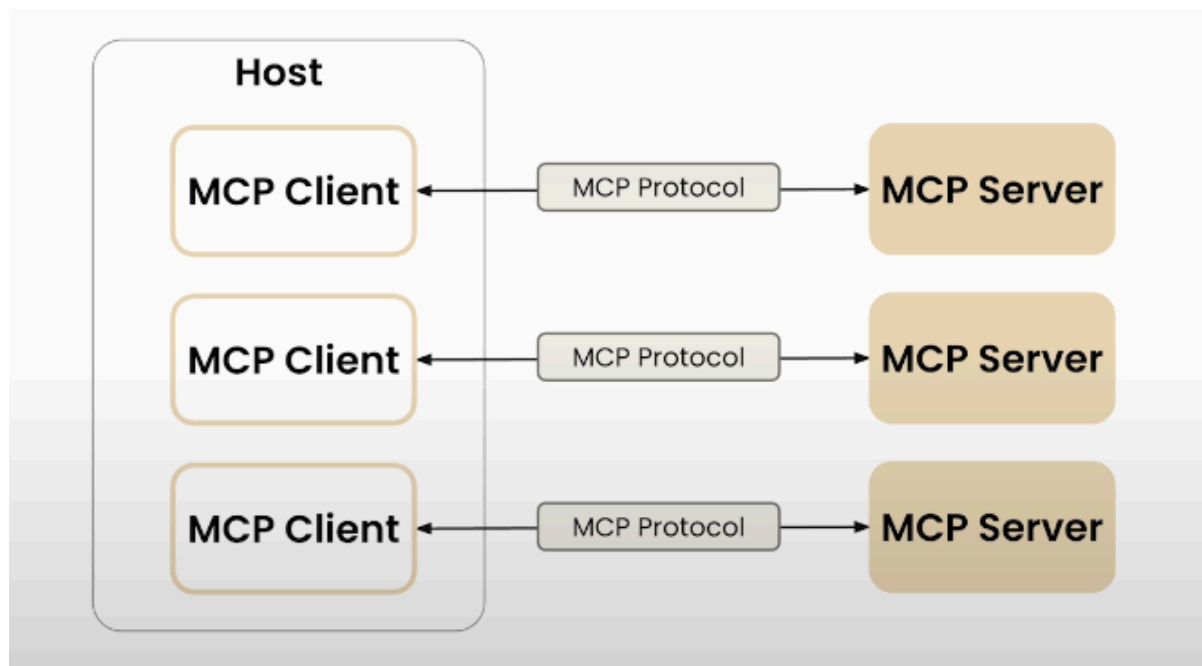
- **Developers:** MCP reduces development time and complexity when building, or integrating with, an AI application or agent.
- **AI applications or agents:** MCP provides access to an ecosystem of data sources, tools and apps which will enhance capabilities and improve the end-user experience.
- **End-users:** MCP results in more capable AI applications or agents which can access your data and take actions on your behalf when necessary.



## MCP are reusable



## Understanding Client Server Architecture in MCP



The MCP architecture consists of three main entities: the **Host**, the **Client**, and the **Server**.

- **MCP Server:**

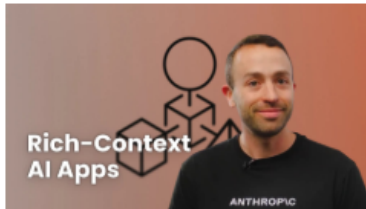
- Servers are **lightweight programs** that expose specific capabilities through the protocol.
- The server's primary job is to **expose information** (tools, resources, and prompts) to the client.

- **MCP Client:**

- MCP clients maintain a **1-to-1 connection** with MCP servers.
- The client's job is to **find resources and find tools**.
- When the client uses a tool (a function provided by the server), the client takes advantage of that tool and executes the necessary data.

- **Host:**

- Clients live inside a host (e.g., Claude desktop or Claude AI).
- The host is an **LLM application** that wishes to access data through MCP.
- The host is responsible for **storing and maintaining** all the clients and their connections to MCP servers.



Short Course • 1 hour 38 mins

## MCP: Build Rich-Context AI Apps with Anthropic

AI Coding

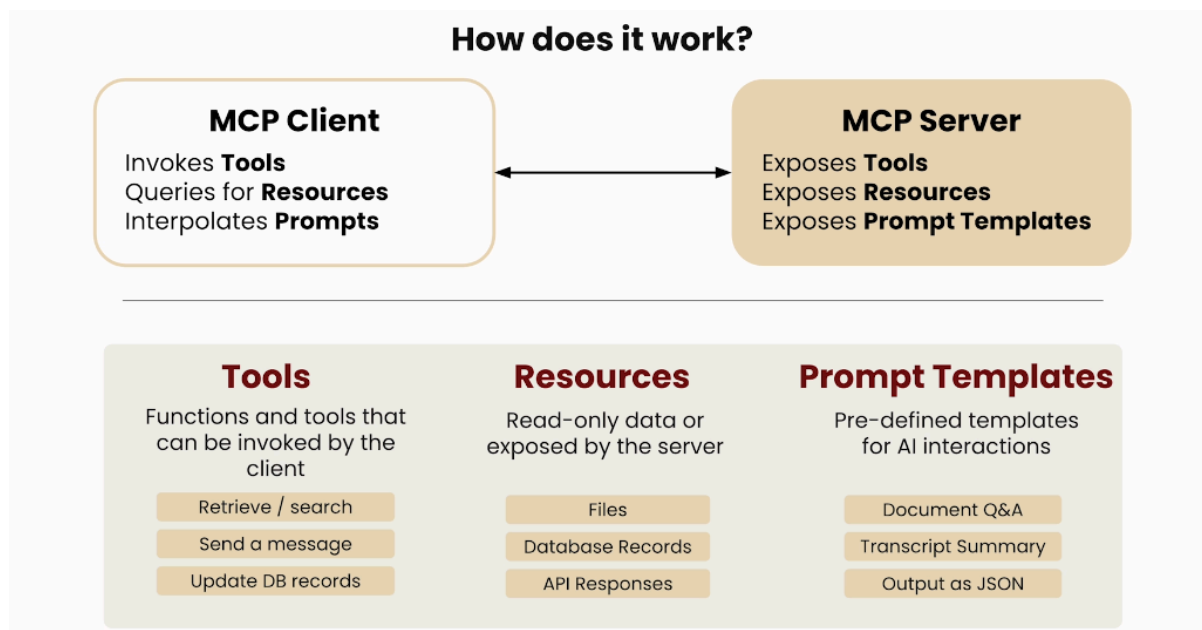
AI Frameworks

Agents

Chatbots

GenAI Applications

# UNDERSTANDING THE CLIENT SERVER ARCHITECTURE



## Defining a Tool

MCP provides SDK's for building servers and clients in a variety of languages. We need to use Python MCP SDK as it is very easy to declare tools

# MCP Server

```
@mcp.tool()
def add(a: int, b: int) -> int:
    """
    Args:
        a: First number to add
        b: Second number to add

    Returns:
        The sum of the two numbers
    """
    return a + b
```

## Resources

- It allows the MCP server to expose data to the client by specifying the URI or a location where the client goes to find the data
- Similar to GET request handlers in a typical HTTP server
- Can return any data type of data- string, JSON, binary etc.
- In the example we have set the 'mime\_type' to give the client a hint as to what data we are returning.
- The resources can be of two types: Direct and Templated

# MCP Server

## *Direct*

```
@mcp.resource(
    "docs://documents",
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
    ↵
```

## *Templated*

```
@mcp.resource(
    "docs://documents/{doc_id}",
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

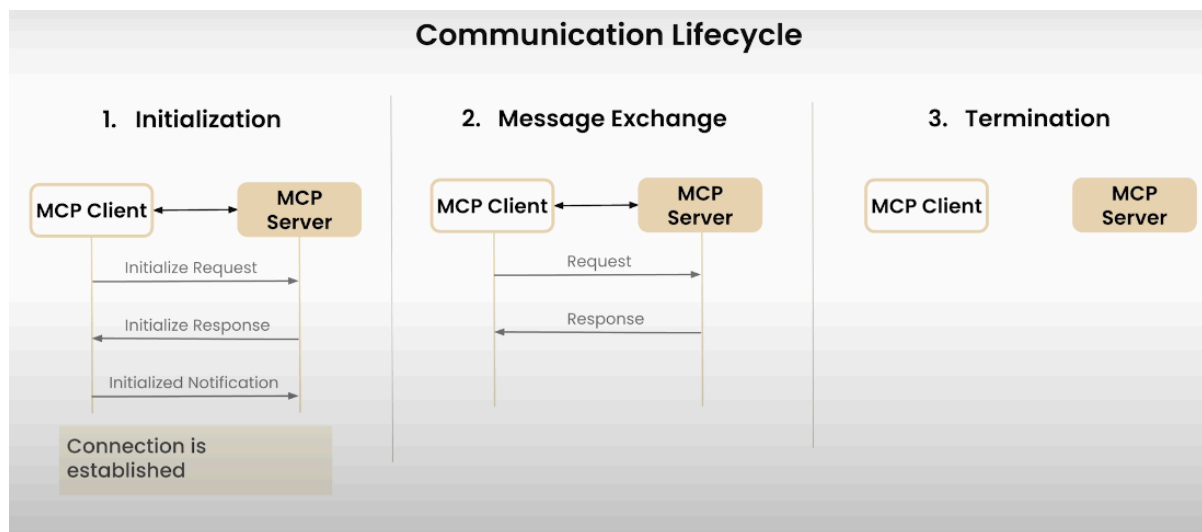
## Prompts

We give a name and description to a prompt and a list of messages/text to return back. It defines a set of user and assistant messages that can be used by the client. These prompts should be high quality and well tested.

# MCP Server

## *Prompt*

```
@mcp.prompt(
    name="format",
    description="Rewrites the contents of
a document in Markdown format",
)
def format_document(
    doc_id: str,
) -> list[base.Message]:
    # Return a list of messages
```



This process consists of three main phases: Initialization, Message Exchange, and Termination.

### 1. Initialization Phase

When the client opens up a connection to the server, the process begins with an initialization phase.

This involves three distinct actions to establish the connection:

1. A request is sent.
2. A response is sent back.
3. A notification is sent to confirm the successful initialization.

Understanding these initialization steps is important because when writing code for MCP, developers will encounter methods, like `initialize`, that correspond to these underlying ideas.

### 2. Message Exchange Phase

Once the connection is successfully initialized, the client and server engage in an exchange of messages. The way these two entities communicate is through messages defined by the Model Context Protocol (MCP) itself.

During this phase:

- Clients can send requests to servers.
- Servers can send requests to clients.
- Notifications can also be sent back and forth between the client and the server.

### 3. Termination Phase

Finally, at the end of the communication, there is a termination of that connection.

### The Role of Transports

Crucial to the mechanics of this communication flow is the transport. A transport handles the underlying mechanism of how messages are sent back and forth between the client and the server, and the choice of transport depends on how the application is running.

**Common transport types include:**

- **Standard IO (Standard Input Output):** Used primarily for local servers where the client launches the server as a subprocess, and the server reads and writes alongside the client using standard in and standard out.
- **HTTP and Server-Sent Events:** Used for remote servers that maintain a stateful connection, meaning the connection remains open between requests, and data is remembered and shared. The server can also send back events and messages to the client.
- **Streamable HTTP:** The recommended transport in updated versions of the protocol, supporting both stateful connections (using HTTP with server events) and efficient stateless connections

(using HTTP on its own). Stateless deployments, where requests are not remembered, are often more efficient for scaling applications.

## PM Perspective: What to Focus On

### Product Strategy

- Identify *when MCP adds real value* (vs. simpler methods)
- Design context-first workflows — where LLMs reason using live data
- Plan for progressive integration — start with read-only, then add tool actions

### Data Privacy & Compliance

- Ensure clear user consent flows
- Restrict sensitive context visibility
- Maintain audit logs for every model request
- Work with legal teams to align on data residency and compliance (e.g., GDPR, HIPAA)

### User Experience Design

- Provide transparency: “Claude is accessing your Notion notes.”
- Let users approve or deny data sources
- Create a unified context dashboard to manage connected apps

### Technical Collaboration

- Work with engineers to define MCP server capabilities (`mcp.json`)
- Ensure your data systems expose clean, structured APIs
- Prioritize server modularity — avoid coupling product logic with LLM client

## Creating an MCP Server

Learnt from here:

<https://learn.deeplearning.ai/courses/mcp-build-rich-context-ai-apps-with-anthropic/lesson/fkbhh/introduction>

### Example Use Case:

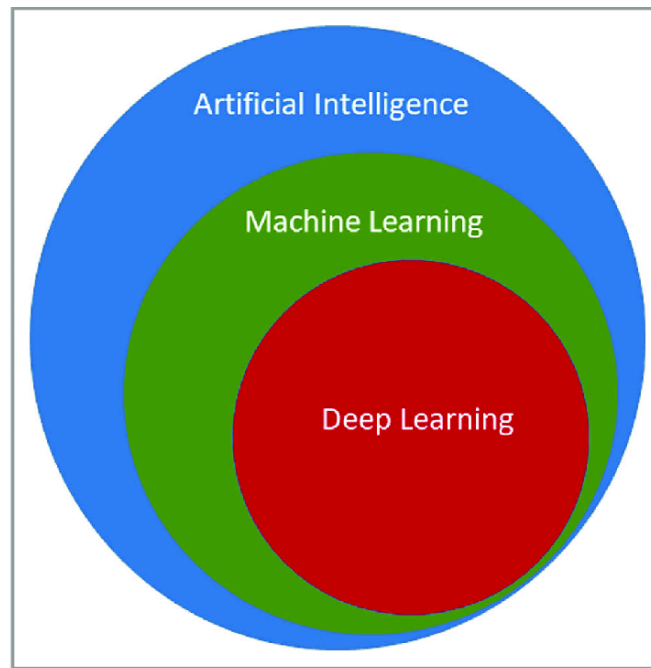
Build an *AI Support Assistant* for a SaaS company using MCP.

How It Works:

1. Connects to Zendesk and internal docs (via MCP servers).
2. User asks: “What’s the latest ticket from customer X?”
3. Model queries the Zendesk MCP server → retrieves ticket.
4. Generates summary and suggests next steps.

5. Logs access and asks for user confirmation before posting a reply.

# What is Artificial Intelligence, Machine Learning and Deep Learning?



## 1. Artificial Intelligence (AI) - The Broad Goal

**Definition:** The science and engineering of making intelligent machines, especially intelligent computer programs. The goal is to simulate human intelligence to perform tasks like problem-solving, decision-making, learning, and planning.

**Scope:** AI includes a vast array of techniques, both modern and old.

**Examples** of AI (Non-ML):

- **Rule-Based Systems (Expert Systems):** Early AI where human experts encoded knowledge into if/then rules (e.g., a simple diagnostic tool).
- **Search Algorithms:** Algorithms used in older video games or route planners (A\* search).

## 2. Machine Learning (ML) - The Modern Method for AI

ML is a subset of AI and is the dominant way to achieve AI today.

**Definition:** The practice of using algorithms to learn patterns from data and improve automatically, without being explicitly programmed for every scenario. Instead of writing rules, you feed the machine data and let it derive the rules.

**Scope:** ML encompasses all methods that rely on data to improve performance.

**Key Techniques:**

- **Supervised Learning** (Classification, Regression)
- **Unsupervised Learning** (Clustering, Dimensionality Reduction)
- **Traditional Algorithms:** Decision Trees, Support Vector Machines (SVMs), Random Forests, K-Means.

### 3. Deep Learning (DL) - A Specialized Type of ML

DL is a specialized subset of ML and is responsible for the recent breakthroughs in AI.

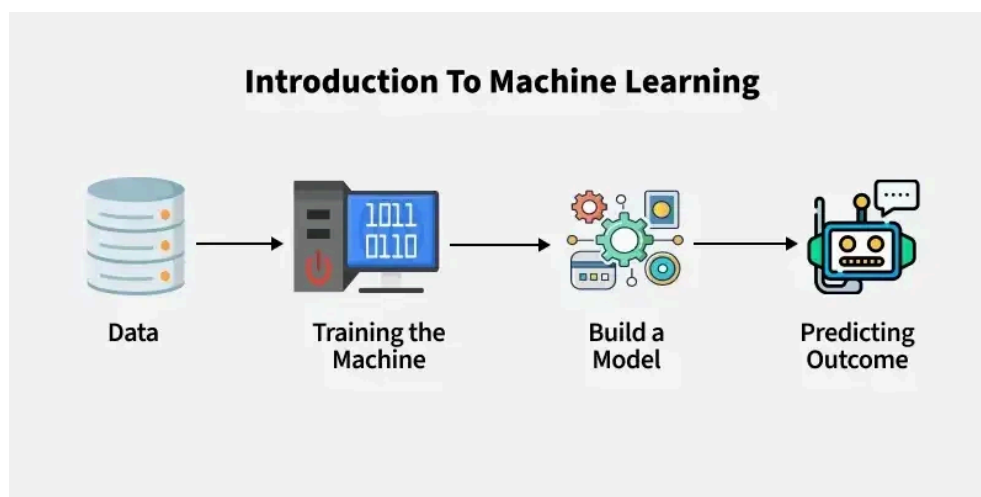
**Definition:** A specific subfield of ML that uses Artificial Neural Networks with multiple processing layers (hence "deep") to learn complex patterns directly from large amounts of raw, unstructured data.

**Distinction from Traditional ML:** Traditional ML often requires a human to manually perform feature engineering (telling the algorithm what to look for, like a car's size). Deep Learning models can automatically extract the relevant features (like edges, shapes, and textures in an image) through its deep layers.

#### Key Techniques:

**Deep Neural Networks:** Including Convolutional Neural Networks (CNNs) for images and Recurrent Neural Networks (RNNs) and Transformers (used in LLMs) for sequence data like text.

**Examples:** Image recognition, natural language processing (like the technology powering large language models), and autonomous driving.



### What is Machine Learning in simpler terms?

Machine learning is a branch of Artificial Intelligence that focuses on developing models and algorithms that let computers learn from data without being explicitly programmed for every task. In simple words, ML teaches the systems to think and understand like humans by learning from the data.

### Why should a PM learn this?

Machine learning (ML) is critically important for an AI Product Manager (AI PM) because it is the fundamental technology underlying most Artificial Intelligence (AI) products. Understanding ML allows the AI PM to effectively define, develop, and manage these products.

Here's a breakdown of why machine learning knowledge is essential for an AI Product Manager:

### **1. Defining and Scoping the Product**

- A solid understanding of ML is crucial for product strategy and roadmapping.
- Feasibility Assessment: The AI PM must determine if a problem is a good fit for an ML solution. This requires knowing the capabilities and limitations of different ML model types (e.g., classification, regression, generative AI).
- Translating Problems: They need to translate customer and business problems (e.g., "reduce customer churn," "improve search relevance") into machine learning tasks (e.g., "a prediction model for churn risk," "a ranking model for search results").
- Data Strategy: ML models are powered by data. The AI PM must understand the required data volume, quality, and labeling effort to achieve a viable product, which directly impacts project scope and timeline.

### **2. Collaboration and Communication**

- The AI PM acts as the translator between business stakeholders and technical teams.
- Working with Data Scientists/Engineers: They must be able to speak the language of the ML team, discussing concepts like model architecture, features, training methodologies, and Model Evaluation Metrics (e.g., accuracy, precision, recall, F1-score).
- Setting Expectations: ML models are probabilistic, meaning their results involve uncertainty and can be unpredictable. The AI PM must manage stakeholder and customer expectations regarding model accuracy, failure modes, and continuous improvement.
- Prioritization: They need to work with the team to prioritize ML features and infrastructure tasks (like data pipelines or MLOps) based on their potential business impact and technical feasibility.

### **3. Product Lifecycle Management**

- AI products have a different lifecycle than traditional software, requiring continuous monitoring and iteration.
- Monitoring Performance: Unlike deterministic software, ML models can degrade over time due to data drift (changes in real-world data). The AI PM must understand and track technical model performance metrics and relate them to business Key Performance Indicators (KPIs).
- Continuous Improvement: They are responsible for implementing feedback loops to gather new data and user input to retrain and improve the model, ensuring the product remains effective.
- A/B Testing and Validation: The AI PM designs and oversees experiments to validate the impact of the ML model on user behavior and business metrics.

### **4. Responsible AI and Ethics**

- Understanding the inner workings of ML is critical for mitigating risks.
- Bias Mitigation: ML models can unintentionally perpetuate or amplify biases present in the training data. The AI PM must be aware of how to identify, monitor, and enforce policies to prevent harmful or unfair outcomes.

- Explainability (XAI): They need to consider how to make the model's decisions transparent or explainable to users and regulators, especially for high-stakes applications (e.g., loan approvals, healthcare).

## **Machine Learning is mainly divided into three core types:**

- Supervised
- Unsupervised
- Reinforcement Learning

### **1. Supervised Learning**

Supervised learning trains a model using labeled data where each input has a known correct output. The model learns by comparing its predictions with these correct answers and improves over time. It is used for both classification and regression problems.

Example: Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients and each patient is labeled as "healthy" or "sick".

| Gender | Age | Label   |
|--------|-----|---------|
| M      | 48  | sick    |
| M      | 67  | sick    |
| F      | 53  | healthy |
| M      | 49  | sick    |
| F      | 32  | healthy |
| M      | 34  | healthy |
| M      | 21  | healthy |

In this example, supervised learning is to use this labeled data to train a model that can predict the label ("healthy" or "sick") for new patients based on their gender and age. For example if a new patient i.e Male with 50 years old visits the clinic, the model can classify whether the patient is "healthy" or "sick" based on the patterns it learned during training.

## 2. Unsupervised Learning:

Unsupervised learning works with unlabeled data where no correct answers or categories are provided. The model's job is to find the data, hidden patterns, similarities or groups on its own. This is useful in scenarios where labeling data is difficult or impossible. Common applications are clustering and association.

Example: Consider the following data regarding patients. The dataset has a unlabeled data where

Only the gender and age of the patients are available with no health status labels.

| Gender | Age |
|--------|-----|
| M      | 48  |
| M      | 67  |
| F      | 53  |
| M      | 49  |
| F      | 34  |
| M      | 21  |

Here unsupervised learning looks for patterns or groups within the data on its own. For example, it might cluster patients by age or gender and group them into categories like "younger healthy patients" or "older patients" without knowing their health status.

## 3. Reinforcement Learning

Reinforcement Learning (RL) trains an agent to make decisions by interacting with an environment. Instead of being told the correct answers, the agent learns by trial and error method and gets rewards for good actions and penalties for bad ones. Over time it develops a strategy to maximize rewards and achieve goals. This approach is

good for problems having sequential decision making such as robotics, gaming and autonomous systems.

Example: While Identifying a Fruit, the system receives an input for example an apple and initially makes an incorrect prediction like "It's a mango". Feedback is provided to correct the error "Wrong! It's an apple" and the system updates its model based on this feedback.

Over time it learns to respond correctly that "It's an apple" when getting similar inputs and also improves accuracy.





## Complete Machine Learning In 6 Hours| Krish Naik

Krish Naik ✓ • 1.5M views

All the materials are available in the below link

<https://github.com/krishnaik06/The-Grand-Complete-Data-Science-...>

### To ML or Not to ML, that is the question!

Implementing ML begins with data. Once enough relevant data is obtained, it is used for predictions, and these predictions are in turn, used for better customer experience, which has the potential to bring more traffic. This traffic again leads to more data. Hence, a cycle is formed.

#### ML can be implemented when your problem:

- Handles very complex logic
- Scales-up fast
- Adapts in real-time
- Requires specialized personalization
- Has existing examples of actual answers

#### Sample ML Problems

1. **Ranking:** Helping users find the most relevant thing  
Example: Ranking algorithm within Amazon search
2. **Recommendation:** Giving users the thing they may be most interested in  
Example: Recommendations from Netflix
3. **Classification:** Figuring out what kind of thing something is  
Example: Product classification for Amazon catalog
4. **Regression:** Predicting a numerical value of a thing  
Example: Predicting sales for specific Amazon products
5. **Clustering:** Putting similar things together  
Example: Related news from Google search
6. **Anomaly:** Finding uncommon things  
Example: Fruit freshness

## SUPERVISED MACHINE LEARNING

**Regression:** This involves predicting a continuous numerical output.

Examples include:

- Predicting house prices based on features like size and location
- Forecasting sales revenue
- Predicting stock prices

|            |                |
|------------|----------------|
| Input(Age) | Output(Weight) |
|------------|----------------|

|    |    |
|----|----|
| 20 | 52 |
| 30 | 69 |
| 56 | 88 |
| 52 | 98 |
| 45 | 45 |
| 49 | 48 |

In the above example: Age is an independent feature(data being trained) and Weight which is the output(data being predicted) is a dependent feature and has continuous variables.

## Linear Regression

Linear regression is one of the simplest machine learning algorithms available, it is used to learn to predict continuous value (dependent variable) based on the features (independent variable) in the training dataset. The value of the dependent variable which represents the effect, is influenced by changes in the value of the independent variable.

| PROS                                                    | CONS                                                                                                                                                                                                                           |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Simple, easy to understand and interpret                | Easy to overfit(when a machine learning model learns its training data too well, capturing not just the underlying patterns but also the noise and random fluctuations, which results in poor performance on new, unseen data) |
| Performs exceptionally well for linearly separable data | Assumes linearity between features and target variable.                                                                                                                                                                        |

**Overfitting-** When the model performs well with training data aka low bias but fails to perform well with test data aka high variance.

**Underfitting:** When model accuracy is bad with training and test data i.e, high bias and high variance

**Generalised Model:** A model with Low bias and low variance

**Cost function:** A cost function in machine learning is a mathematical equation that measures the difference between a model's predictions and the actual, correct values

## Ridge Regression vs Lasso Regression

Ridge and Lasso Regression are two popular techniques in machine learning used for regularizing linear models to avoid overfitting and improve predictive performance. Both methods add a penalty term to the model's cost function to constrain the coefficients, but they differ in how they apply this penalty.

Ridge Regression, also known as L2 regularization, adds the squared magnitude of the coefficients as a penalty. On the other hand, Lasso Regression, or L1 regularization, introduces a penalty based on the absolute value of the coefficients.

### **What is Ridge Regression or (L2 Regularization) Method?**

Ridge regression, also known as L2 regularization, is a technique used in linear regression to prevent overfitting by adding a penalty term to the loss function or  $\lambda$ (Hyperparameter). This penalty is proportional to the square of the magnitude of the coefficients (weights).

### **What is Lasso Regression or (L1 Regularization) Method?**

Lasso regression, also known as L1 regularization, is a linear regression technique that adds a penalty to the loss function or  $\lambda$ (Hyperparameter) to prevent overfitting and also helps in feature selection. This penalty is based on the absolute values of the coefficients.

### **When to Use Ridge Regression?**

Ridge Regression is most suitable when all predictors are expected to contribute to the outcome and none should be excluded from the model. It reduces overfitting by shrinking the coefficients, ensuring they don't become too large, while still keeping all the predictors in the model.

For example, when predicting house prices, features like size, number of bedrooms, location, and year built are all likely relevant. Ridge Regression ensures these features remain in the model but with reduced influence to create a balanced and robust prediction.

### **When to Use Lasso Regression?**

Lasso Regression is ideal when you suspect that only a few predictors are truly important, and the rest may add noise or redundancy. It performs automatic feature selection by shrinking the coefficients of less important predictors to zero, effectively removing them from the model.

For example, in genetic research, where thousands of genes are analyzed for their effect on a disease, Lasso Regression helps by identifying only the most impactful genes and ignoring the irrelevant ones, leading to a simpler and more interpretable model.

**Classification:** This involves predicting a categorical output.

Examples include:

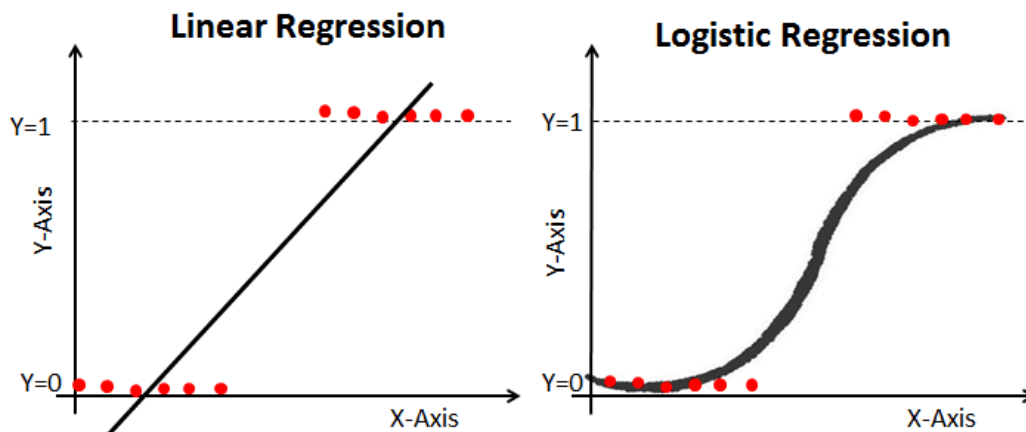
- Spam detection (classifying emails as spam or not spam)
- Image recognition (identifying objects in images)
- Medical diagnosis (classifying a patient's condition)

| Marks 1 | Marks 2 | Marks 3 | Result |
|---------|---------|---------|--------|
| 20      | 44      | 41      | Pass   |
| 30      | 63      | 3       | fail   |
| 55      | 22      | 5       | fail   |

In this example we have 3 independent features(data being trained) marks1, marks 2 and marks 3 and one dependent feature which is the output(data being predicted). Here the output no. of category i.e, binary category as it results in Pass or fail.

## Logistic Regression

Logistic Regression is a special case of Linear Regression where target variable (y) is discrete / categorical such as 1 or 0, True or False, Yes or No, Default or No Default. A log of the odds is used as the dependent variable. Using a logit function, logistic regression makes predictions about the probability that a binary event will occur.



| PROS                                     | CONS                                                  |
|------------------------------------------|-------------------------------------------------------|
| Simple, easy to understand and interpret | Overfitting                                           |
| Well-calibrated for output probabilities | Experience difficulty capturing complex relationships |

## Naive Bayes Classifiers

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorization and sentiment analysis. It is named as "Naive" because it assumes the presence of one feature does not affect other features. The core intuition behind Naive Bayes is that it works based on the principles of **Bayes theorem** and **probability theory**.

### Foundation in Bayes Theorem

The Bayes theorem is derived from the concept of conditional probability. The fundamental formula for Bayes theorem, derived from the equivalence of  $P(A \text{ and } B)$  and  $P(B \text{ and } A)$ , is:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

This theorem is based on dependent events, where the outcome of one event affects the probability of another.

Let's take a simple example:

| Email | Contains "Free" | Contains "Win" | Spam / Not Spam |
|-------|-----------------|----------------|-----------------|
| 1     | Yes             | Yes            | Spam            |
| 2     | Yes             | No             | Spam            |
| 3     | No              | Yes            | Not Spam        |
| 4     | No              | No             | Not Spam        |

We need to predict whether a **new email that contains "Free" and "Win"** is spam or not.

Step 1: Calculate Prior Probabilities

Total emails = 4

Spam = 2, Not Spam = 2

$P(\text{Spam}) = 2/4 = 0.5$   $P(\text{NotSpam}) = 2/4 = 0.5$

Step 2: Calculate Likelihoods

| Feature | $P(\text{feature} = \text{Yes} \mid \text{Spam})$ | $P(\text{feature} = \text{Yes} \mid \text{NotSpam})$ |
|---------|---------------------------------------------------|------------------------------------------------------|
| "Free"  | $2/2 = 1.0$                                       | $0/2 = 0.0$                                          |
| "Win"   | $1/2 = 0.5$                                       | $1/2 = 0.5$                                          |

Step 3: Calculate Posterior Probabilities

For new email = "Free = Yes, Win = Yes"

For Spam:

$P(\text{Spam} \mid \text{Free, Win}) = P(\text{Free} \mid \text{Spam}) * P(\text{Win} \mid \text{Spam}) * P(\text{Spam}) = (1.0) * (0.5) * (0.5) = 0.25 = (1.0) * (0.5) * (0.5) = 0.25 = (1.0) * (0.5) * (0.5) = 0.25$

For Not Spam:

$P(\text{NotSpam} \mid \text{Free, Win}) = P(\text{Free} \mid \text{NotSpam}) * P(\text{Win} \mid \text{NotSpam}) * P(\text{NotSpam}) = (0.0) * (0.5) * (0.5) = 0 = (0.0) * (0.5) * (0.5) = 0 = (0.0) * (0.5) * (0.5) = 0$

**Conclusion:**

Since  $0.25 > 0$ , the email is classified as **spam**

## K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification (and sometimes regression). It's a lazy learning algorithm — meaning it doesn't actually learn a model during training. Instead, it stores the training data and makes predictions only when a new data point comes in.

KNN fundamentally operates based on the concept of finding the **K nearest neighbor** points to a new data point.

## Core Mechanism

The core steps for making a prediction using KNN are:

1. **Determine K:** The value  $K$  is a **hyper parameter** that specifies the number of nearest closest points the algorithm will consider. Researchers often test a range of  $K$  values (e.g.,  $K=1$  to 50) and evaluate the error rate to select the best model.

2. **Calculate Distance:** The algorithm determines which points are "nearest" by calculating the distance between the new point and the existing data points. Two primary distance metrics are used:

◦ **Euclidean Distance:** Calculates the distance between two points(  $X_1, Y_1$  and  $X_2, Y_2$  ) using the formula:

$$\sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

**Manhattan Distance:** Calculates the distance by summing the absolute unit movement in the coordinates (not the hypotenuse distance). The formula is:  $|X_2 - X_1| + |Y_2 - Y_1|$

## Prediction for Different Problem Types

The algorithm uses the calculated distances to make a prediction based on the nature of the problem:

| Problem Type          | Prediction Method      | Detail                                                                                                                                                 |
|-----------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Classification</b> | <b>Majority Voting</b> | The algorithm categorizes the new data point into the class from which the <b>maximum number of points are coming</b> among the $K$ nearest neighbors. |
| <b>Regression</b>     | <b>Averaging</b>       | The algorithm calculates the <b>average of all the points</b> among the $K$ nearest neighbors, and this average value becomes the predicted output.    |

## Decision Tree

Decision Tree (DT) is a powerful supervised machine learning algorithm capable of solving both classification and regression problems. It uses a visual structure, similar to a flowchart, to represent a series of nested if-else conditions.

A Decision Tree is considered a white box model because its internal logic—how decisions are made and how features are split—can be easily visualized and understood.

## Core Structure and Splitting

**The algorithm is built upon a hierarchical structure of nodes:**

1. **Root Node:** This is the starting point of the decision process.
2. **Internal Nodes:** These nodes represent conditions (like if age is less than or equal to 18) based on specific features. Based on the condition being true (Yes) or false (No), the data is split into branches.
3. **Leaf Nodes (or Pure Nodes):** These are terminal nodes where no further splitting occurs, and they contain the final prediction (output).

## Decision Tree Classification Process

Classification problems rely on determining the purity of a split. The goal is to continue splitting impure nodes until pure leaf nodes are obtained.

- **Pure Split/Node:** A node is pure if it contains instances belonging exclusively to one class (e.g., all "Yes" outputs or all "No" outputs). Once a node is pure, it does not need to split further.
- **Impure Split/Node:** A node is impure if it contains a mix of different classes (e.g., 2 "Yes" and 3 "No"). Impure nodes require further splitting using other features.

## Ensemble Bagging And Boosting Techniques

### What Are Ensemble Techniques?

Imagine you're hiring for your product team.

Would you trust one person's opinion about a candidate, or would you prefer to ask multiple experts and then make a collective decision?

That's what ensemble techniques do. They combine multiple models to make better, more stable predictions.

| Type            | How They Work                                                                                                                                           | Real-World Analogy                                                                                      |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Bagging</b>  | Models work <b>in parallel</b> — each model learns from a slightly different slice of data. Then, their predictions are combined (e.g., majority vote). | Like a <b>panel of interviewers</b> independently scoring a candidate, then you take the average score. |
| <b>Boosting</b> | Models work <b>in sequence</b> — each model learns from the mistakes of the previous one.                                                               | Like a <b>mentor-student system</b> — each new mentor helps fix what the last one missed.               |

### Bagging (Bootstrap Aggregating)

Idea: Reduce overfitting by training multiple versions of the same model on different random samples of the data.

How It Works:

- Take your full dataset.
- Randomly sample parts of it (some data points repeat, some are left out).
- Train separate models (e.g., decision trees) on each sample — all in parallel.
- When predicting, all models vote or average their answers.

**Key Algorithm:**

**Random Forest:** It's basically "a forest of decision trees," each seeing different parts of the data and features. The combined prediction is usually more stable and accurate than any single tree.

**PM Analogy:**

Imagine each decision tree is a product reviewer.

Each one reads different customer reviews and gives a score.

You then take the average score → a more balanced, less biased decision.

**Boosting**

**Idea:** Turn weak models into a strong one by learning from mistakes.

How It Works:

- Train a simple model (weak learner) — maybe a tiny decision tree.
- See which data points it got wrong.
- Give those difficult cases more weight.
- Train the next model, focusing on those mistakes.
- Repeat many times — each new model "boosts" the last one.

**Key Algorithms:**

AdaBoost (Adaptive Boosting) — adjusts weights on wrong predictions.

Gradient Boost / XGBoost — improves efficiency and accuracy by mathematically minimizing errors.

PM Analogy:

Think of a sales coaching program:

Coach 1 trains your reps, but they still make mistakes.

Coach 2 focuses on the reps who struggled most.

Coach 3 helps fix what the first two couldn't.

By the end, your sales team (the combined effort) performs far better.

## UNSUPERVISED MACHINE LEARNING

### Clustering

Clustering is an unsupervised machine learning technique that groups unlabeled data into clusters based on similarity. Its goal is to discover patterns or relationships within the data without any prior knowledge of categories or labels.

- Groups data points that share similar features or characteristics.
- Helps find natural groupings in raw, unclassified data.
- Commonly used for customer segmentation, anomaly detection and data organization.
- Works purely from the input data without any output labels.
- Enables understanding of data structure for further analysis or decision-making.
- Some common clustering algorithms:

## **K-Means Clustering**

K-Means is one of the most commonly used algorithms in unsupervised machine learning, especially for customer segmentation, behavior analysis, personalization, and market research.

### **As a PM, understanding it helps you:**

- Know how your users or products can be grouped naturally
- Drive data-informed decisions (like personalized campaigns, pricing tiers, or UX design improvements)
- Communicate intelligently with data scientists and analysts

What is K-Means Clustering?

K-Means is an unsupervised algorithm — meaning there are no pre-labeled answers or categories.

It finds patterns in the data by grouping similar points together into “clusters.”

Think of it as a way for your data to self-organize into meaningful groups — like users forming natural segments based on their behavior.

### **How K-Means Works — PM Edition**

Let's imagine you're a PM at a fitness app, and you have users with two attributes: Workout frequency (days per week) and Daily calorie burn

You want to group them into distinct segments for targeted offers. Here's how K-Means works:

Step 1: Choose the number of clusters (K)

You decide how many groups you want (say, 3 clusters = beginner, intermediate, advanced users).

Step 2: Initialize centroids

The algorithm picks 3 random “center points” (centroids) in the data space.

Step 3: Assign users to nearest centroid

Each user is assigned to the closest center, creating temporary groups.

Step 4: Update centroids

The algorithm recalculates the center of each cluster based on the current members.

Step 5: Repeat

It keeps re-assigning users and re-calculating centers until the clusters stabilize (i.e., centroids stop moving much).

Final result → Distinct groups of users with similar behavior.

## Hierarchical Clustering

Hierarchical Clustering is an unsupervised learning technique used to group similar data points into clusters. Unlike K-Means, it does not require you to specify the number of clusters (K) beforehand.

It instead creates a hierarchy (tree-like structure) of clusters called a dendrogram.

### Two Types of Hierarchical Clustering

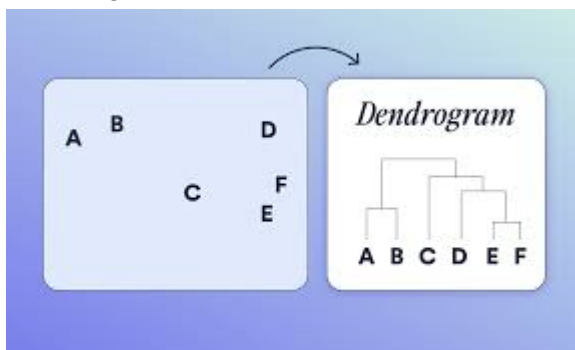
1. Agglomerative (Bottom-Up)
  - Most commonly used
  - Start with each point as its own cluster
  - Gradually merge the closest clusters until only one remains
2. Divisive (Top-Down)
  - Start with all data in one cluster
  - Gradually split the cluster into smaller ones

### Steps in Agglomerative Hierarchical Clustering

Let's say we have 5 points: A, B, C, D, E

- Step 1: Compute the distance (similarity) between every pair of points (usually Euclidean distance).
- Step 2: Merge the two points that are closest together (say A & B).
- Step 3: Recalculate distances — treat (A,B) as a single cluster.
- Step 4: Merge the next two closest clusters.
- Step 5: Repeat until all points are in one cluster.

### Dendrogram Example



Each merge point represents the distance between the clusters being joined.

By drawing a horizontal line through the dendrogram, you can “cut” it to decide how many clusters to form.

## Silhouette Clustering- Validating Clusters

The Silhouette Score is a metric that measures how well data points are clustered — it evaluates both cohesion (compactness within a cluster) and separation (distance between clusters).

It answers two key questions:

- Are points well matched to their assigned cluster?
- Are clusters well separated from each other?
- 

### **Why It's Important**

Silhouette Score provides a quantitative validation metric for clustering models that have no ground truth (unsupervised learning).

It helps ensure that clusters are:

- Cohesive (points are close within the same cluster)
- Well-separated (far from other clusters)

## **DBSCAN**

DBSCAN is an unsupervised clustering algorithm that groups together data points that are close to each other (dense regions) and marks points that lie far away as outliers or noise.

Unlike K-Means, it does not require you to predefine the number of clusters (K) — instead, it lets the data shape its own clusters.

### **Why It's Useful**

- Imagine you're analyzing customer location data on a map:
- Some areas have dense clusters of users (e.g., city centers).
- Some have sparse or random points (e.g., rural customers or data errors).
- DBSCAN automatically detects the dense areas as clusters and ignores outliers — without you having to decide how many groups there should be.

# Introduction to Agent Workflow



## **What Is an Agentic AI Workflow?**

An agentic AI workflow is a process in which an AI system, most commonly leveraging Large Language Models (LLMs), executes a series of steps—often iteratively—to complete a task. This is distinct from the classic, linear approach where a user prompts an LLM once for an entire output (e.g., writing an essay) and receives a single response. Agentic workflows are designed to more closely mimic complex human problem-solving by involving multiple discrete stages: planning, researching, synthesizing, reviewing, and revising.

### **Key Characteristics**

- **Iterative Execution:** The workflow involves multiple interdependent steps, typically looping through reflection and improvement stages.
- **Task Decomposition:** Complex tasks are broken down into smaller, manageable components—mirroring human strategies for complex work.
- **Varying Autonomy:** The workflow can be scripted (less autonomous) or grant the AI greater decision-making power (more autonomous).

## **Task Decomposition and Workflow Steps**

Task decomposition is the practice of dissecting complex tasks into atomic steps the AI (or supporting tools/APIs) can execute.

### **Detailed Steps in Agentic Workflows**

- **Step 1:** Identify the complex job and break it into specific, sequential subtasks.
- **Step 2:** Assign each subtask to an LLM, specialized AI model, or external tool/API suited to the task (e.g., PDF-to-text conversion for invoice extraction).
- **Step 3:** Sequence the steps so output from one subtask flows to the next.
- **Step 4:** Refine and iterate—decompose further if a step is not easily automatable.

### **Examples:**

| Use Case        | High-Level Steps                                                               |
|-----------------|--------------------------------------------------------------------------------|
| Essay Writing   | Outline → Search web → Analyze sources → Draft → Reflect → Revise → (Optional) |
| Invoice Process | PDF to text → Document type check → Field                                      |

|                  |                                                                     |
|------------------|---------------------------------------------------------------------|
|                  | extraction → Database update                                        |
| Customer Service | Extract info → Query database → Draft reply → Queue for review/send |

Iteration is crucial: If initial outputs are unsatisfactory, add additional steps (such as reflection or human-in-the-loop verification).

### Degrees of Autonomy in Agentic AI

Autonomy in agentic workflows is not binary; it lies on a spectrum.

#### 1. Less Autonomous Agents

- Steps strictly predefined by developers
- Functions/tools are hard-coded (limited to set APIs or databases)
- Useful for well-structured tasks like basic invoice processing or order status queries

#### 2. Semi-Autonomous Agents

- AI has limited discretion over chosen tools and step sequencing
- Steps and tools are predefined but AI may decide which to use for a given query

#### 3. Highly Autonomous Agents

- Significant decision-making delegated to the AI
- The LLM plans and sequences steps dynamically, often choosing tools and gathering more context as needed
- May even generate new functions or propose workflow extensions
- Suitable for advanced research, creative work, and multi-stage customer service cases

### Benefits of Agentic AI Workflows

Agentic workflows deliver measurable advantages, especially in complex or knowledge-intensive domains.

- **Higher Quality and Performance:** Iterative, decomposed processes enable more thoughtful, accurate, and robust outputs. As shown in coding evaluations, iterative agentic prompts can outperform newer LLMs running non-agentic, single-command prompts.
- **Human-Like Iteration:** Mimics natural human work (think-plan-write-revise), greatly improving trustworthiness and reliability.
- **Parallelization:** Some subtasks (like web search or data retrieval) can run simultaneously, outperforming human speed for scalable workloads.
- **Modularity:** Components (tools, models, steps) can be swapped or optimized individually. E.g., swapping web search engines or integrating better LLMs for distinct parts of the workflow.

### Application Areas for Agentic AI

Agentic AI is seeing practical deployment in a variety of domains:

#### 1. Invoice Processing

- Automates extraction and system entry of structured data from invoices.
- Involves PDF conversion, field verification, database update—all decomposed for optimal automation.

#### 2. Customer Service Agents

- *Basic inquiries:* Structured flows for order status, shipment tracking, etc.

- *Complex queries*: Dynamic planning for tasks like inventory checks, returns, and policy exceptions—areas where step-sequencing is not obvious ahead of time.
3. Research Agents
    - Multi-step research (search, synthesize, draft, review) for tasks like whitepapers, compliance reviews, or market analysis.
    - Yields deeper, more reliable insights than linear prompting.
  4. Computer Use by Agents
    - Agents act as digital workers: controlling a browser, filling forms, navigating UI to perform goal-based tasks (e.g., booking flights).
    - Still an emerging area, with reliability challenges (slow sites, unexpected layouts).

## Evaluating Agentic AI Workflows ("AI Evals")

Rigorous evaluation ("evals") is essential to gauge, debug, and improve agentic workflows.

### Metrics and Approaches

- **Objective Evals**: Easily codified checks (e.g., "Does the output mention a competitor's name?"). Automated code scans are used to flag these.
- **Subjective Evals**: Where outputs are evaluated for nuance (e.g., essay quality), use a separate judging LLM or human review. Prompt the judge with a quality scoring rubric (e.g., 1-5 scale).
- **End-to-End Evals**: Assess overall system output—does the agent fulfill the end user's goal?
- **Component-Level Evals**: Judge the output of a single workflow step.
- **Error Analysis**: Manually or automatically inspect intermediate traces to spot flaws or improvement areas.

Iterative improvement is key: Evals help prioritize workflow modifications and drive performance up over time.

## Agentic AI Design Patterns

Four core design patterns shape agentic workflows:

1. Reflection
  - AI reviews and critiques its own outputs, feeding back to improve results (common in code or essay generation).
2. Tool Use
  - AI is empowered to call external tools or APIs: web search, code execution, database queries, etc.
  - The LLM decides which tool to use for which step.
3. Planning
  - AI autonomously determines the workflow sequencing for unstructured requests, planning step-by-step action chains.
  - E.g., a "Hugging GPT" style agent plans multiple APIs to fulfill one complex objective.
4. Multi-Agent Collaboration
  - Multiple simulated agents work together, each with a specialized role: e.g., researcher, writer, editor, or test/critique agent for software.
  - Collaboration between specialized agents often yields more thoughtful and robust outputs for multi-stage tasks.

These insights form the technical and conceptual backbone for building, deploying, and managing state-of-the-art agentic AI workflows in modern product management and beyond.

## **Workflows vs Agents:**

Workflows are systems where LLMs and tools are orchestrated through predefined code paths.

Agents, on the other hand, are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.